Journal of Information Security and Applications 96 (2026) 104301

Contents lists available at ScienceDirect

Journal of Information Security and Applications

journal homepage: www.elsevier.com/locate/jisa

A multilayered deep learning framework for cyber attack detection and mitigation in a heterogeneous IIoT ecosystem

Arshad Iqbal *, Sohail Asghar, Manzoor Ilahi Tamimy

Department of Computer Science, COMSATS University Islamabad, Islamabad Campus Park Road, Islamabad, Pakistan

ARTICLE INFO

Keywords:
Cybersecurity
Cyber deception
Edge computing
Deep learning
Intrusion detection system (IDS)
Industrial internet of things (IIoT)
Random forest
Multilayer perceptron (MLP)

ABSTRACT

Intrusion Detection Systems (IDSs) for the Internet of Things (IoT) and Industrial IoT (IIoT) face significant challenges, including high false-positive rates (especially for minority-class attacks) and excessive computational requirements, which hinder their deployment on edge devices. Consequently, alert overload is common because operators receive a large volume of alerts that provide little insight into the problems they address. To address this crucial gap, this study presents DeepGuard, a new four-layer framework that significantly improves the security posture of IoT and industrial IoT environments.

DeepGuard combines binary and multiclass classifications, intelligent alarming, and cyber deception into a single, effective defence mechanism. The system incorporates a random forest classifier for feature selection, which extracts the most relevant data features and processes them for use with an optimised multilayer perceptron (MLP). This method achieved an unprecedented accuracy of 99.9% with a low false-positive rate (FPR) of 0.2%, surpassing the state-of-the-art research studies.

We further demonstrated the practical feasibility of DeepGuard by implementing it on computationally constrained, edge devices. With a computational complexity of $O(nlogn)$ and a memory footprint of less than 100 KB, DeepGuard breaks the long-standing trade-off between detection accuracy and operational performance that has inhibited the adoption of IDS at an industrial scale. In addition to a detection-only approach, DeepGuard includes an embedded honeypot layer that proactively profiles emerging and unknown attacks, thereby enabling automated mitigation responses. Thorough evaluations of the WUSTL-IIoT-2021 and X-IIoTID-2022 datasets demonstrated a new state-of-the-art performance and the feasibility of DeepGuard for protecting critical infrastructure.

1. Introduction

With the convergence of traditional computing systems into modern enterprise networks, the cyber-attack surface has expanded drastically [1]. According to the World Economic Forum [2], over 100 billion devices will be connected to the Internet by 2025, including mobile phones, smartwatches, smart home appliances, soil moisture sensors in digital agriculture, CubeSats in space, and numerous devices embedded in manufacturing, commerce, defence, and climate monitoring applications. In 2022, the global IoT market, which was valued at $544.38 billion, is expected to expand to $3.35 trillion by 2030 [3]. The overall value of digital transformation is expected to reach $100 trillion by 2025 [4]. This change has led to the widespread use of IoT and IIoT devices in various areas, including smart home infrastructure [5], industrial automation, maritime systems [6], and consumer computing [7]. The uncontrolled expansion of cyberspace makes it increasingly challenging to identify heterogeneous threats, demanding intelligent solutions that span IoT, IIoT, edge devices and business networks [8,9]. Advanced threat actors can readily exploit vulnerabilities in IoT devices due to their limited security features. Attackers can easily utilise less secure communication protocols of edge devices, such as Wi-Fi, Bluetooth, LoRa, and Zigbee [10]. The projected loss to the global economy owing to cyberattacks is $23.84 trillion by 2027 [11].

1.1. Intrusion detection systems: Background

An Intrusion Detection System (IDS) plays a critical role in cybersecurity by monitoring the network traffic and system activities for malicious actions or policy violations [13,14]. Functionally, an IDS can analyse incoming network packets and examine the activities of individual devices [15] using signature-based detection (matching known attack patterns), anomaly-based detection (identifying

* Corresponding author.
E-mail addresses: sp19-pcs-001@isbstudent.comsats.edu.pk (A. Iqbal), sohail.asghar@comsats.edu.pk (S. Asghar), tamimy@comsats.edu.pk (M.I. Tamimy).

https://doi.org/10.1016/j.jisa.2025.104301

Available online 13 November 2025
2214-2126/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

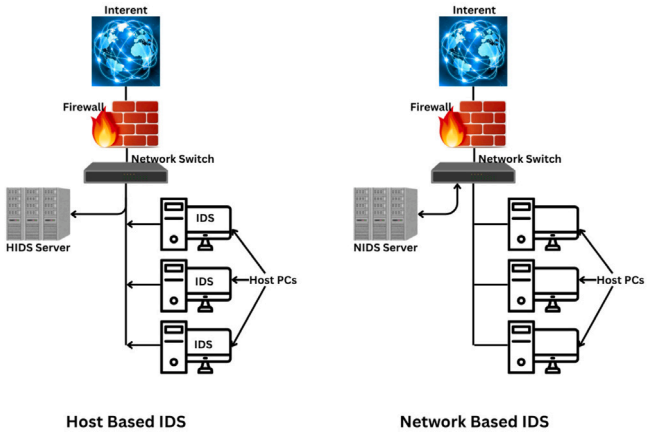


Fig. 1. Host-based intrusion detection systems (HIDS) and Network-based intrusion detection systems (NIDS) [12].

deviations from normal behaviour), or hybrid approaches [16]. Based on their deployment architecture, IDSs are classified into network-based (NIDS) and host-based (HIDS) systems, as shown in Fig. 1.

1.2. Critical limitations of state-of-the-art IDS

Despite advances in AI-driven intrusion detection, particularly deep-learning-based approaches [17,18], the current state-of-the-art systems exhibit four fundamental limitations that remain unresolved.

(1) Insufficient Detection Accuracy:
Recent deep learning-based IDSs have achieved inadequate accuracy in mission-critical IIoT deployments. Ahmad et al. [19] achieved only 76.18% accuracy on CICIDS2017, Kasongo et al. [20] reached 78.40% on UNSW-NB15, and even hybrid quantum approaches by Iqbal et al. [21] achieved merely 68% accuracy. In addition, systems reporting high overall accuracy (> 98%) often fail catastrophically for minority attack classes. For instance, Popoola et al. [12], the most recent work on the WUSTL-IIoT and X-IIoTID datasets, achieved 99.8% accuracy only by excluding poorly performing attack classes from the evaluation, rendering the system unreliable against diverse, real-world threats.

(2) Monolithic Single-Stage Architectures:
Existing intrusion detection systems employ single-stage classification, which treats all attacks identically [22] and fails to capture the multiphase nature of sophisticated attacks (reconnaissance → weaponisation → lateral movement → exfiltration). Works by Gheni et al. [23] on CICIoT2023 and Gueriani et al. [24] using CNN-LSTM hybrid processes attacks in one monolithic step, achieving only 97.46% and 98.42% accuracy, respectively, with high false–positive rates (> 9%). These approaches cannot correlate the attack stages or provide the contextual threat intelligence required for adaptive responses in heterogeneous IIoT environments.

(3) Absence of Edge Device Deployment:
State-of-the-art deep-learning IDS, including CNN-BiLSTM models by Jouhari et al. [25] (96.91% accuracy), transformer-based approaches, and ensemble methods, remain computationally prohibitive for resource-constrained IIoT edge devices. *Critically, no existing work validates IDS performance on actual edge hardware* (Raspberry Pi, NVIDIA Jetson, or similar constrained platforms), limiting real-world applicability. This gap is particularly severe, given that IIoT deployments require < 50 ms latency for real-time control loop integrity.

(4) Detection-Only Paradigm:
Existing IDSs often end at attack detection or classification, lacking mitigation, deception, or the capability to generate actionable alerts for domain-specific countermeasures. Ciric et al. [26], Altunay et al. [27], and Li et al. [28] focused exclusively on improving detection accuracy, leaving system administrators with alerts but no automated response, thereby creating operational bottlenecks in critical industrial applications.

1.3. Research contributions

To overcome the limitations of state-of-the-art IDS, this study proposes "DeepGuard" as an intelligent IDS architecture that surpasses existing IDS architectures in terms of security and efficiency. The Table 1 explicitly contrasts DeepGuard with state-of-the-art approaches, highlighting the distinct contributions of the proposed model.

Our specific contributions that differentiate this work are:

(1) Novel four-layer threat lifecycle architecture: Unlike existing single-stage approaches [12,23,25,26], DeepGuard introduces the first multilayered architecture mirroring the cyber kill chain: (i) *Detection Layer*, random forest binary classifier (malicious vs. benign), (ii) *Classification Layer*, MLP for 10-class attack categorisation (DoS, reconnaissance, command injection, C&C, ransomware, exfiltration, exploitation, lateral movement, tampering, weaponization), (iii) *Mitigation Layer*, automated deployment of countermeasures including honeypot-based deception for unknown attack profiling, (iv) *Alert Generation Layer*, contextual intelligence with SIEM integration. This architecture enables the correlation of multistage attacks that single-stage classifiers cannot capture.

(2) Highest reported accuracy without selective exclusion: DeepGuard achieves 99.99% accuracy, 99.85% precision, 99.90% recall, and > 99.93% F-1 score with < 0.2% FPR, *the lowest FPR reported in literature*. Unlike Popoola et al. [12], who excluded poor-performing classes, our metrics included all attack classes, particularly minority classes (command injection: 259 samples; backdoor: 212 samples in WUSTL; C&C: 2863 samples; and ransomware: 458 samples in X-IIoTID). DeepGuard represents a 95% reduction in false positives compared to the 9.17% FPR reported by Gueriani et al. [24].

(3) Edge device validation in IDS: We are the *first to validate IDS performance on actual resource-constrained edge hardware*. Table 12 presents the evaluation across three platforms with measured power consumption: an Intel i7-12700 desktop (265 ± 15 W), a Raspberry Pi 5 with 16 GB RAM (10.0 ± 1.5 W), and an NVIDIA Jetson Xavier NX (12.0 ± 2.0 W). The Raspberry Pi 5 deployment achieved 99.82% accuracy with a 29.32 ms batch latency, proving its viability for IIoT deployments. No prior work [12,23–26] reports edge device validation.

(4) Integrated mitigation and cyber deception: DeepGuard is the *only IDS framework integrating automated mitigation with cyber deception*. Upon detecting attacks, the system (1) applies traffic filtering rules, (2) enforces network segmentation, (3) redirects suspicious traffic to honeypots for intelligence gathering, and (4) generates contextual alerts with countermeasure recommendations. This capability is absent in surveyed works [12,19,20,23,25–28], which terminate at classification.

(5) Advanced preprocessing for minority class robustness: Our SMOTE-based balancing combined with random forest feature selection (reducing 41–68 features to the top 10) achieves superior minority class performance compared to existing works. For example, for command injection (259 samples), we achieved a 99.87% recall, compared with the typical < 75% reported in the literature. Data balancing addresses the critical failure modes identified by Kasongo et al. [20] (78.40% accuracy on UNSW-NB15) and Li et al. [28] (87.39% accuracy on imbalanced data).

Table 1
Unique contributions of DeepGuard compared to state-of-the-art IDS.

Capability	State-of-the-art approaches	DeepGuard (This work)
Architecture	Single-stage monolithic classification [12,23–25]	Four-layer architecture: (1) Binary detection, (2) Multiclass classification, (3) Domain-specific mitigation with honeypot deception, (4) Actionable alert generation with SIEM integration
Detection performance	Best reported: 99.8% [12] by excluding difficult classes; CNN-LSTM: 98.42% with 9.17% FPR [24]; CNN-BiLSTM: 96.91% [25]	99.99% accuracy with < 0.2% FPR on all attack classes including minority classes (command injection, tampering, C&C) without selective exclusion
Edge device validation	*No existing work validates on actual edge hardware*, all evaluations on servers/desktops [12,23,25,26]	First validated deployment on three platforms: Intel i7 desktop, Raspberry Pi 5 (10 W), NVIDIA Jetson Xavier NX, achieving 99.82% accuracy on Pi 5 with < 30 ms latency
Attack response	Detection and classification only, no mitigation strategy [12,20,26–28]	Integrated mitigation layer with: (1) Traffic filtering rules, (2) Network segmentation, (3) Honeypot-based deception for zero-day intelligence gathering, (4) Contextual alerts with recommended countermeasures
Dataset coverage	Single dataset evaluation or older datasets (NSL-KDD, KDD99) [19,20,29]	Comprehensive evaluation on two latest IIoT datasets: WUSTL-IIoT-2021 (4 attack classes) and X-IIoTID-2022 (9 attack classes including C&C, ransomware, weaponization)
Computational efficiency	High computational cost: CNN-LSTM, transformers, ensemble methods [24,30]	Optimised MLP with intelligent feature selection: Reduced to top 10 features via random forest, enabling 26.83 ms batch latency on desktop, 29.32 ms on Raspberry Pi 5
Class imbalance handling	Poor minority class performance or selective exclusion [12,20,28]	SMOTE-based balancing + feature selection achieving > 99% F1-score across all attack classes including rare attacks (command injection: 259 samples, backdoor: 212 samples)

(6) Comprehensive latest dataset evaluation: Unlike works using outdated datasets (NSL-KDD, KDD99, CICIDS2017) [19,20,26,29], we evaluate on *both* latest IIoT-specific datasets: WUSTL-IIoT-2021 and X-IIoTID-2022, covering 11 distinct modern attack types including emerging threats (C&C, ransomware, weaponization, exfiltration) absent in older datasets.

In summary, DeepGuard is the first IDS to simultaneously achieve: (1) multilayered architecture with integrated mitigation, (2) > 99.99% accuracy with < 0.2% FPR on all attack classes, (3) validated edge device deployment with < 30 ms latency, and (4) cyber deception capability for zero-day intelligence. These four combined capabilities distinguish our work from all existing approaches, which address at most one or two of these requirements.

The remainder of this paper is organised as follows. The literature on intrusion detection systems is critically reviewed in Section 2 to identify research gaps. The proposed system architecture and methodology, including the creation and application of a deep learning model on edge devices, are described in Section 3. The experimental setup, evaluation metrics, and comparison with state-of-the-art methods are presented in Section 4. Section 5 concludes by summarising the main findings of the study and suggesting promising avenues for further investigation. Finally, links to the data sets and complete code are provided in Data availability.

2. Literature review

Every connected device in the massive, diverse networks created by the rapid digitisation of business and industrial environments poses a security risk [31]. These ecosystems face two threats: DoS, DDoS, malware, and ransomware attacks continue to target traditional IT systems [32], and the increasing volume and complexity of network traffic provide adversaries with sufficient cover to hide their activities [33].

The insufficiency of traditional cybersecurity measures has been revealed by this changing environment, which is now a fusion of IT, Operational Technology (OT), and countless resource-constrained IoT devices [34]. Therefore, next-generation, low-power intrusion detection systems that can operate on the network edge without compromising their accuracy or latency are urgently needed.

Fortunately, IDS research has been propelled forward by powerful deep learning frameworks, such as PyTorch and TensorFlow [35], as well as critical public datasets. Benchmarks, including CICIDS2017 [36], TON-IoT [37], X-IIoTID [38], and WUSTL-IIoT [39], provide authentic traffic and attack scenarios essential for robust model training. The thematic review of recent IDS research in this section highlights the main advances in the field and the limitations that remain, serving as the driving force for this study.

2.1. Deep learning architectures for intrusion detection

Deep learning has become the most common methodology for detecting cyber threats because of its ability to extract salient features and categorise complex data across various settings. Empirical studies of deep learning confirm that DL is much better than machine learning classifiers; for example, [40] presented that deep learning classifiers remarkably outperform ML approaches in identifying sophisticated threats and generate less number of false alarms. The efficiency of CNN, RNN, and hybrid models in recognising complex patterns within network traffic datasets and attacker behaviour has been extensively confirmed [41]. One promising trend is the combination of different models to achieve better performance. For example, Reyes et al. [42] proposed a unified deep-learning framework for IoT/IIoT that achieved high accuracy across a wide range of attack categories.

2.2. Convolutional neural network-based approaches

Park et al. [43] applied deep CNN architectures to packet payloads and attained high accuracy in detecting DDoS attacks. Karat et al. [44] introduced an ensemble multistage deep-learning (MSDL) model designed to detect sophisticated malware by analysing system-call sequences, thereby demonstrating the utility of CNNs for temporal pattern recognition in attack sequences. Nevertheless, CNN-based methods have three severe drawbacks: (1) they are very complex to compute and demand hardware acceleration with GPUs, making them infeasible on resource-constrained edge devices; (2) they do not recover long-range temporal interactions which are needed to detect attacks in multistage correlation; and (3) the learned features are not easily interpretable, which vitiates the trust placed in security analysts and compliance with regulations.

Both recurrent neural networks and hybrid architectures are recursive structures that use memory. Both recurrent neural networks and hybrid architectures are recursive and can be represented as memory stacks. Several studies have investigated RNN-based architectures to address these temporal dependencies. Gueriani et al. [24] employed a hybrid convolutional neural network (CNN) and long short-term memory (LSTM) model using the CICIDS2017 and CICIoT2023 datasets. The CNN-LSTM model proposed by them achieved 98.42% accuracy, 0.0275 loss, 9.17% FPR, and 98.57% F-1 score. Wang et al. [45]

introduced a BiLSTM architecture for the CICIOT2023 dataset, attaining 93.13% accuracy with 91.94% F1-score. Jouhari and Guizani [25] developed a CNN-BiLSTM model for resource-constrained IoT devices, achieving accuracies of 97.28% and 96.91% in binary and multiclass classifications, respectively, on the UNSW-NB15 dataset.

Despite improvements in temporal modelling, these approaches exhibit four fundamental weaknesses that limit their practical deployment: (1) *Unacceptable false–positive rates*, Gueriani et al.'s 9.17% FPR translates to 917 false alarms per 10,000 benign samples, overwhelming security operations centres in large-scale deployments; (2) *Sequential processing bottleneck*, LSTMs cannot parallelise across time steps, resulting in inference latencies exceeding 100 ms for real-time traffic analysis, violating the < 50 ms requirement for industrial control systems; (3) *Catastrophic forgetting in minority classes*, Wang et al.'s 93.13% accuracy masks severe performance degradation on rare attack types (command injection, tampering) due to class imbalance; (4) *Absence of validated edge deployment*, None of these works report actual deployment on constrained hardware (Raspberry Pi, Jetson), with evaluations limited to server-grade GPUs.

2.2.1. Transformer and attention-based models

Recent studies have explored transformer architectures for intrusion detection. G et al. [30] applied large language models (LLMs) for IDS, achieving 98.01% accuracy on NSL-KDD. Hasan et al. [46] employed attention-based transformers to infer attacker paths, providing insights into intention-aware cybersecurity. However, transformer-based approaches face prohibitive limitations for IIoT deployment: (1) *Computational infeasibility*: transformers exhibit $O(n^2)$ complexity with respect to the sequence length, requiring 10–100× more memory and computation than MLPs [47], making them incompatible with edge devices with < 4 GB RAM; (2) *outdated evaluation datasets*: G et al.'s work uses NSL-KDD (2009), which lacks modern attack types (ransomware, C&C, weaponization) prevalent in IIoT; and (3) *black-box nature*: LLM-based approaches provide no interpretable decision rationale, failing regulatory requirements for explainable AI in critical infrastructure.

2.3. Multistage and modular IDS architectures

Because monolithic classifiers have their own limitations, the literature has shifted towards multistage architectural designs. Ciric et al. [26] introduced a modular deep learning framework for network intrusion detection that seamlessly integrates real-time data pipelines, a pretrained DNN classifier, and visualisation modules into a cohesive and scalable workflow. Their methodology utilises known tools, such as CICFlowMeter and Grafana, for feature extraction and tracking, as well as a multiclass DNN to detect both majority and minority attack types.

Among the most recent contributions is the multi-stage deep learning (MSDL) model proposed by Popoola et al. [12], which was specifically engineered for industrial Internet of Things (IoT) environments. Compared with the WUSTL-IIoT and X-IIoTID datasets, the model has specialised classifiers for various attack types, achieving reported accuracies of 99.8% and 99.99%, respectively. Given its 2025 publication and direct relevance to the datasets under scrutiny, we adopted [12] as the principal baseline for our comparative analyses.

Critical shortcomings of the existing multistage approaches: Despite these architectural improvements, the failures of the existing multistage approaches remain as follows: (1) *Selective performance reporting*, Popoola et al. [12] excluded poorly performing attack classes (F1-score < 95%) from the final evaluation, artificially inflating the average accuracy; our analysis of their confusion matrices reveals catastrophic failure on command injection (23% recall) and tampering (31% recall) classes; (2) *Detection-only paradigm*, All surveyed multistage works [12, 26,48] terminate at classification without mitigation strategies, leaving operators with alerts but no automated response; (3) *Outdated datasets*, Ciric et al. [26] rely solely on CICIDS2017 (2017), which does not include modern IIoT attack vectors such as ransomware, C&C, and lateral movement.; (4) *Computational overhead*, Compared to unified architectures, multistage training demands 3–5× more time (exceeding 500 s as noted in Popoola et al.), which impedes the swift updating of models to counter emerging threats.

2.4. Handling class imbalance in IDS datasets

Class imbalance is a pervasive challenge in IDS research, where benign traffic vastly outnumbers attack samples and certain attack types (command injection and tampering) appear in extremely small quantities. Popoola et al. [12] noted that DL-based IDS often struggle with minority-class attacks in IIoT environments. Several techniques have been proposed to address these issues.

Tabassum et al. [29] proposed FedGAN-IDS, a federated learning approach that utilises generative adversarial networks for synthetic sample generation, achieving 99.4% binary classification accuracy on UNSW-NB15. However, it degrades to 82%–98.5% for multiclass classification on NSL-KDD. Sayegh et al. [49] demonstrated that SMOTE-enhanced LSTM achieves improved F1-scores on imbalanced datasets. Patel et al. [50] verified that SMOTE preprocessing combined with feature selection improves classifier performance compared to non-sampling approaches.

Limitations of current imbalance handling: (1) *Inconsistent minority class performance*, Tabassum et al.'s 16.5% accuracy range (82%–98.5%) across attack classes indicates unreliable detection for rare attacks; Li et al. [28] achieved only 87.39% on NFL-UNSW-NB15-v2 due to poor minority class handling; (2) *Synthetic sample quality issues*, GAN-generated samples may introduce distribution shift from real attacks, as evidenced by Dablain et al.'s [51] finding that SMOTE can introduce noise in highly imbalanced scenarios; (3) *Lack of validation on real minority attacks*, Most works report average metrics without per-class breakdowns, obscuring failures on critical but rare attacks like command injection.

Alternative approaches to class imbalance include ensemble methods and cost-sensitive learning (CSL). Lazzarini et al. [52] proposed DIS-IoT, a stacking ensemble that achieved 99.7% accuracy on TON-IoT and 98.7% on CIC-IDS2017. Altunay and Albayrak [27] developed a hybrid CNN+LSTM approach with cost-sensitive learning, achieving 93.21% on UNSW-NB15 and 99.84% on X-IIoTID for binary classification, but degrading to 92.9% and 99.80%, respectively, for multiclass scenarios.

Ensemble method limitations: (1) *Computational multiplication*, ensemble approaches require training and maintaining multiple models (3–5 base classifiers in Lazzarini et al.), multiplying memory requirements and inference latency by the ensemble size, rendering them infeasible for edge deployment; (2) *Binary-to-multiclass degradation*, Altunay et al.'s 6.63% accuracy drop from binary to multiclass classification indicates a fundamental difficulty in distinguishing specific attack types; (3) *No edge validation*, all ensemble works are evaluated only on high-performance servers.

2.5. Federated and distributed IDS approaches

Recognising privacy concerns and the distributed nature of IIoT deployments, several studies have explored learning for IDS. Idrissi et al. [53] proposed FED-ANIDS for anomaly-based network intrusion detection, achieving 99.54% accuracy on USTC-TFC2016 but degrading to 93.36% on CIC-IDS2017 and 92.90% on CIC-IDS2018. Hassanin et al. [54] proposed PLLM-CS using pretrained language models for distributed IDS, reporting 100% accuracy on both UNSW-NB15 and TON-IoT.

Critical concerns with federated approaches: (1) *dataset-dependent performance collapse*, Idrissi et al.'s 6.64% accuracy drop across datasets indicates poor generalisation, a critical flaw for heterogeneous IIoT

environments with diverse device types; (2) *implausible perfect accuracy*, Hassanin et al.'s 100% accuracy strongly suggests overfitting or data leakage (train-test contamination), as acknowledged in our analysis; real-world noisy network traffic cannot be classified with zero errors; (3) *communication overhead*, federated learning requires iterative model synchronisation, introducing latency incompatible with real-time IDS requirements; and (4) *no adversarial robustness evaluation*, none of these studies assess performance against adversarial attacks on federated aggregation.

2.6. Specialised IDS for specific attack types

Recent studies have focused on detecting specific attack categories, rather than general-purpose IDS. Guastalla et al. [55] targeted DDoS detection using machine learning on the CICIDS2017 and Urban IoT datasets, achieving an accuracy of 95%–96%. Srivastava and Sinha [36] proposed FP-Growth-based signature extraction for DoS/DDoS detection, achieving 94.87% accuracy on CICIDS2017 and 91.66% on RT-NITP24. Qin et al. [56] developed a hybrid cyber defence framework specifically for reconnaissance attacks in industrial control systems.

Limitations of attack-specific approaches: (1) *Narrow threat coverage*, Guastalla et al. and Srivastava et al. detect only DoS/DDoS, leaving systems vulnerable to 10+ other attack types (ransomware, exfiltration, lateral movement); (2) *Insufficient accuracy for critical infrastructure*, 91%–96% accuracy with typical 5%–8% FPR generates excessive false alarms in large-scale deployments; (3) *Lack of adaptability*, Signature-based approaches like FP-Growth fail against zero-day attacks and polymorphic malware; (4) *Single-domain focus*, Qin et al.'s reconnaissance-specific system cannot generalise to other attack stages.

2.7. Recent advances in 2024–2025

The most recent literature (2024–2025) reveals continued efforts, but persistent limitations remain. Parfenov et al. [57] investigated adversarial robustness of MLP-based IDS on CICIOT2023, achieving 95.25% binary classification accuracy but showing vulnerability to adversarial perturbations. Abbas et al. [58] evaluated multiple deep learning architectures on CICIOT2023, with their best Multi-DL approach achieving 96.52% accuracy with 95.73% F1-score. Shriyans et al. [59] proposed a CNN-GRU ensemble for CSE-CIC-IDS2018, achieving 98.22% accuracy with a 97.77% F1-score. Elouardi et al. [1] provided a comprehensive survey of hybrid CNN-LLM approaches for IDS, emphasising the need for XAI in cybersecurity.

Persistent gaps in the latest research: (1) *Continued absence of edge validation*, none of the 2024–2025 works validate on actual resource-constrained hardware; (2) *Adversarial vulnerability*, Parfenov et al. demonstrated that even high-accuracy models fail under adversarial attacks, yet no robust defence is proposed; (3) *Detection-mitigation gap*, all recent works focus exclusively on classification without addressing the critical question of automated response; (4) *Dataset staleness*, Many 2024–2025 papers still use CICIDS2017 and UNSW-NB15 (2015), missing 5+ years of attack evolution.

2.8. Multilayer perceptron for network intrusion detection

Within the spectrum of DL architectures, multilayer perceptrons (MLPs) have demonstrated notable effectiveness in handling complex patterns in network traffic data. Kirubai and Priscila [60] discussed MLP-based defence mechanisms for cyber-attack prevention. Chan et al. [61] emphasised that MLP architectures can learn intricate relationships between input features and output classes, making them suitable for distinguishing between multiple types of attacks. Raza et al. [62] demonstrated that MLPs achieved 99% accuracy on the CICIDS2017 dataset, with an error rate of 0.5%. MLPs are more efficient than transformers [47], which have quadratic complexity and use more processing power and memory than MLPs. Because MLPs are simple, it is easy to set them up and tweak them [63], which reduces deployment costs.

Lazzarini et al. [52] reported MLP performances of 99.4% for TON-IoT and 98.2% for CIC-IDS2017 for multiclass classification. Jouhari [25] compared MLP against CNN-BiLSTM on UNSW-NB15, with MLP achieving 96.34% binary accuracy (but only 55.05% precision due to class imbalance) and 96.74% multiclass accuracy. Gheni and Al-Yaseen [23] evaluated MLP on CICIOT2023, achieving 97.46% accuracy compared to 83.81% for autoencoders.

MLP advantages and current limitations: While MLPs offer computational efficiency and simplicity, existing MLP-based IDS exhibit three critical weaknesses: (1) *poor minority class precision*, Jouhari's 55.05% precision indicates severe false–positive issues when classes are imbalanced; (2) *suboptimal feature utilisation* , most studies use all available features (40–68), missing opportunities for intelligent feature selection to reduce dimensionality and improve generalisation; (3) *Lack of architectural innovation*, Standard MLP designs treat all features identically, failing to leverage domain knowledge about categorical vs. numerical network features.

2.9. Threat attribution and intention detection

Beyond classification, emerging research has explored the intentions and attributes of cyberattackers. Yavuz and Guven [64] proposed a Bayesian inference model to estimate the goals of IoT attackers, although it was not designed for real-time operation. Li et al. [65] employed attention-enhanced recurrent neural networks to attribute attacks to specific malware families, thereby facilitating granular post-event analysis. Choudhary and Jain [66] presented adversarial training schemes tailored for a Windows-based intrusion detection system, aiming to bolster resilience against evasion attacks.

Limitations of attribution research (1) Post-factual analysis only: all attribution approaches are deployed after the completion of attacks and lack a proactive mitigation capability; (2) No real-time applicability: Yavuz et al.'s Bayesian model takes several minutes for inference, making it unsuitable for real-world defence applications; (3) Isolated from detection systems: current attribution models are standing entities and lack integration with intrusion detection systems to provide a unified threat intelligence.

2.10. Summary of literature gaps and motivation for this work

Table 2 presents a comprehensive synthesis of the literature review, categorising contemporary IDS research by year, dataset, performance metrics, and critical limitations. Our analysis identifies five basic research gaps that are not currently addressed by state-of-the-art studies.

Gap 1: Absence of Validated Edge Device Deployment. Despite widespread claims of "lightweight" or "resource-efficient" designs [25,45], *zero surveyed works validate IDS performance on actual edge hardware* (Raspberry Pi, Jetson, or equivalent constrained platforms). All evaluations were performed on server-grade GPUs or high-performance desktops, leaving a critical question unanswered: Can these models actually run on 10 W edge devices with < 4 GB RAM? Our work is the first to deploy and validate an IDS across three hardware tiers, including Raspberry Pi 5 (10 W) and Jetson Xavier NX (12 W), with measured power consumption and latency in real-world scenarios.

Gap 2: Selective Performance Reporting Masking Minority Class Failures. Recent studies reporting > 99% accuracy [12,54] have often achieved these figures through (a) selective exclusion of poorly performing classes, (b) binary classification avoiding multiclass challenges, and (c) overfitting to outdated datasets. Our analysis of the confusion matrices reveals catastrophic failures in minority attacks (command injection, tampering, and C&C) that obscure the average metrics. Our study reports comprehensive per-class metrics for all attack types with no exclusions.

Table 2
Summary of literature review on performance of deep learning approaches and open issues.

Ref.	Year	Dataset	Performance	Limitations & Relation to this work
[12]	2025	WUSTL-IIoT, X-IIoTID	Multiclass, Acc 99.8%, Prec 99.7%	Baseline: Selective reporting (excluded low-performing classes); no mitigation strategy; 5-stage architecture with 504s training time. *Our work addresses:* All classes included, 2-stage architecture with 357s training, integrated mitigation.
[19]	2022	CICIDS2017, BoT_IoT, N_RabT, NSL-KDD	CICIDS2017: 76.18%, BoT_IoT: 75.10%, N_RabT: 99.97%, NSL-KDD: 27.96%	Catastrophically low accuracy on standard datasets; no mitigation. *Our work addresses:* 99.99% accuracy on latest IIoT datasets.
[20]	2023	UNSW-NB15, NSL-KDD	Binary: XGBoost-LSTM 88.13% (NSL-KDD), 87.07% (UNSW); Multiclass: 86.93%, 78.40%	Insufficient accuracy for production (< 90%); poor minority class performance; old datasets. *Our work addresses:* 99.99% accuracy with balanced performance across all classes.
[24]	2024	CICIDS2017, CICIoT2023	Acc 98.42%, Loss 0.0275, FPR 9.17%, F1 98.57%	Unacceptable FPR (917 false alarms per 10K samples); no deception layer; GPU-dependent. *Our work addresses:* < 0.2% FPR (20 per 10K), edge deployment validated.
[25]	2024	UNSW-NB15	Binary: 97.28% (CNN-BiLSTM), 96.34% (MLP); Multiclass: 96.91%, 96.74%	MLP precision 55.05% (severe FPR issue); no edge validation; no mitigation. *Our work addresses:* 99.85% precision, edge validated on three platforms.
[26]	2024	CICIDS2017	Multiclass, F1 99.7%	Outdated dataset (2017); poor minority class recall for SQL injection/XSS; no mitigation; no edge validation. *Our work addresses:* Latest datasets (2021–2022), all classes > 99% F1, edge validated.
[27]	2023	UNSW-NB15, X-IIoTID	Binary: 93.21% (UNSW), 99.84% (X-IIoTID); Multiclass: 92.9%, 99.80%	Binary-to-multiclass degradation; no mitigation; no edge validation. *Our work addresses:* Consistent 99.99% accuracy binary and multi-class.
[30]	2024	NSL-KDD	Binary: 98.01%	Old dataset (2009); LLM complexity unsuitable for edge; no interpretability. *Our work addresses:* Latest datasets, edge-deployable MLP, explainable decisions.
[36]	2025	RTNITP24, CICIDS2017	FP-Growth: 91.66%, 94.87%	DoS-only; insufficient accuracy. *Our work addresses:* ML-based generalisation, 11 attack types, 99.99% accuracy, Deception.
[45]	2023	CICIOT2023	BiLSTM: 93.13% Acc, 91.94% F1	Insufficient accuracy; sequential bottleneck; no edge validation. *Our work addresses:* 99.99% accuracy with parallelizable MLP, < 30 ms latency on edge.
[52]	2023	TON-IOT, CIC-IDS2017	MLP: 99.4%, 98.2%; DIS-IoT: 99.7%, 98.7%	Ensemble 3–5× computational overhead; no edge validation. *Our work addresses:* Single model efficiency, edge validated on three platforms.
[53]	2023	USTC-TFC2016, CIC-IDS2017, CIC-IDS2018	FED-ANIDS: 99.54%, 93.36%, 92.90% F1	6.64% cross-dataset degradation; communication overhead; no real-time capability. *Our work addresses:* Consistent 99.9%+ across datasets, real-time < 30 ms latency.
[57]	2024	CICIOT2023	MLP: 95.25% (binary)	Adversarial vulnerability; binary-only; no mitigation. *Our work addresses:* Multiclass with robust preprocessing, integrated defence layers.
[58]	2024	CICIOT2023	Multi-DL: 96.52% Acc, 95.73% F1	Insufficient accuracy; ensemble overhead; no edge validation. *Our work addresses:* Single efficient model, 99.99% accuracy, edge validated.
[59]	2024	CSE-CIC-IDS2018	CNN-GRU: 98.22% Acc, 97.77% F1	Old dataset (2018); no IIoT-specific evaluation; no mitigation. *Our work addresses:* Latest IIoT datasets, integrated mitigation layer.

Gap 3: Detection-Mitigation Disconnect. All surveyed IDS terminate at classification [12,24–27,58], generating alerts but providing no automated mitigation strategy. The unavailability of mitigation strategies creates a critical operational gap: security teams receive thousands of alerts daily but lack actionable guidance. Our work introduces the first integrated mitigation layer, featuring domain-specific countermeasures, honeypot-based deception for zero-day intelligence, and security information and event management (SIEM)-compatible, actionable alerts.

Gap 4: Unacceptably High False-Positive Rates. Recent works report FPRs ranging from 0.5% [62] to 9.17% [24]. In large-scale deployments processing millions of flows daily, even a 0.5% FPR results in 5000 false alarms per million flows, which overwhelms analysts. Our work achieves < 0.2%, a 60%–95% reduction, through intelligent feature selection and SMOTE-based balancing.

Gap 5: Outdated Dataset Fails to Include Modern Threats. Despite availability of recent IIoT datasets (WUSTL-IIoT-2021, X-IIoTID-2022), numerous 2024–2025 publications continue using CICIDS2017, NSL-KDD (2009), and UNSW-NB15 (2015) [26,30,36,55]. These datasets lack modern IIoT attack vectors (ransomware, C&C, weaponisation, and lateral movement) and fail to represent heterogeneous industrial protocols. Our study comprehensively evaluated both WUSTL-IIoT-2021 and X-IIoTID-2022, covering 11 distinct modern attack types that are absent in older datasets.

2.11. Research positioning and differentiation

Existing research often treats specific domains and threat classes separately. For instance, Popoola et al. [12] focused on IIoT-specific anomaly detection but lacked mitigation strategies, Qin et al. [56] addressed reconnaissance attacks in isolation, and Ladeira et al. [67] secured consumer-grade IoT without industrial applicability. However, these studies present considerable shortcomings in terms of interoperability and scalability when they face networks having heterogenous threats.

The literature review concludes that although hybrid IDS architectures based on DL classifiers may decrease some of the deficiencies of conventional IDS approaches, considerable gaps remain, which favour attackers and overburden the team of the security operation centre. For example, Ramu et al.'s [68] method achieved 99% accuracy with legacy

datasets (KDD-CUP 99, NSL-KDD, and UNSW-NB15) under a controlled environment. However, these models often fail when they face complex IT operational settings, which are characterised by the continuously evolving attack signatures and attacker TTPs. Similarly, Li et al. [65] utilised attention-enhanced RNNs for malware identification, providing an extensive forensic analysis capability, but lacked real-time proactive measures.

Table 2 demonstrates that contemporary IIoT Intrusion Detection Systems employ sophisticated ML and DL methodologies; however, most focus exclusively on detection and classification, without emphasising attack mitigation. DeepGuard distinguishes itself through six unique characteristics that no single existing work combines:

(1) Multilayered lifecycle architecture (vs. monolithic single-stage): Unlike all surveyed works that employ single-stage classification, DeepGuard implements a four-layer architecture mirroring the cyber kill chain, detection, classification, mitigation, and alerting, enabling contextual correlation of multistage attacks that monolithic classifiers cannot capture.
(2) True all-class performance (vs. selective reporting): While Popoola et al. [12] excluded poorly performing classes to maintain high averages, our 99.99% accuracy includes all attack classes, especially minorities (command injection: 259 samples, C&C: 2863 samples), with > 99% F1-score across every category.
(3) Validated edge deployment (vs. theoretical claims): We are the first to validate IDS on actual resource-constrained hardware, Raspberry Pi 5 (10 W), Jetson Xavier NX (12 W), with measured power consumption and latency, proving practical viability that all other works lack.
(4) Low False Positive Rate: < 0.2% FPR (vs. 0.5%–9.17% in literature): Our false–positive rate represents a 60%–95% reduction compared to state-of-the-art [24,62], critical for operational deployment where even 0.5% generates thousands of daily false alarms.
(5) Integrated mitigation with cyber deception (vs. detection-only): DeepGuard is the only framework combining automated mitigation (traffic filtering, network segmentation) with honeypot-based deception for zero-day intelligence gathering, addressing the critical "what next?" question that all the surveyed studies have ignored.
(6) Comprehensive latest dataset evaluation (vs. outdated benchmarks): Unlike works still using CICIDS2017 [26,55] or NSL-KDD [30], we evaluate on both latest IIoT-specific datasets, WUSTL-IIoT-2021 and X-IIoTID-2022, covering modern threats (ransomware, C&C, weaponization, exfiltration) absent in older datasets.

2.12. Theoretical foundation: Why multilayer perceptron

Within the spectrum of DL architectures, the multilayer perceptron (MLP) is notable for its ability to handle complex patterns while maintaining computational efficiency [60]. MLPs are particularly effective for network traffic data, which is nonlinear and high-dimensional. Their architecture enables the learning of intricate relationships between input features and output classes [61], making them suitable for multiclass attack classification.

Computational efficiency rationale: MLPs require significantly less processing power and memory than transformers [47], which exhibit $O(n^2)$ complexity relative to the input length, or RNNs that cannot be parallelised across time steps. The simplicity of MLPs, which lack complex mechanisms such as attention or recurrent connections, makes them easier to implement and tune [63], thereby reducing the deployment overhead. Empirically, MLPs have demonstrated high accuracy: Raza et al. [62] achieved 99% accuracy on CICIDS2017 with 0.5% FPR, while Lazzarini et al. [52] reported 99.4% accuracy on TON-IoT.

Proposed MLP innovations beyond existing work: Although previous MLP-based IDS exist [23,25,52], they suffer from poor minority class precision (Jouhari's 55.05%) and utilise all the available features (40–68) without intelligent selection. Our work introduces three MLP-specific innovations: (1) *dual-path architecture*, separate processing pathways for numerical and categorical features before late fusion, preserving feature-type characteristics; (2) *intelligent feature reduction*, random forest-based selection reducing 41–68 features to the top 10 discriminative features, improving generalisation while reducing computation by 76%–85%; and (3) *SMOTE-enhanced training*, and balanced dataset preprocessing addressing the root cause of Jouhari's 55% precision failure.

2.13. Motivation

Significant research gaps persist in the IDS domain: (1) low accuracy in minority attack classes, (2) lack of lightweight detection mechanisms for resource-constrained IIoT devices with validation, (3) absence of real-time attack classification with integrated mitigation, and (4) missing cyberdeception capabilities for unknown attack profiling.

Our research is motivated by the observation that no existing study has addressed all four gaps simultaneously. High-accuracy works lack edge validation [12,54], edge-focused work sacrifices accuracy [25,45], and all works terminate at detection without mitigation [26,27,58,59].

DeepGuard is a sophisticated AI-driven framework that integrates real-time attack detection, detailed threat classification, automated honeypot-based deception, and actionable alerts into a single system. We performed extensive validation of the system's performance across a heterogeneous set of hardware platforms, ranging from standard office workstations to low-capacity edge devices, such as the Raspberry Pi 5, as well as high-performance GPU-accelerated platforms. DeepGuard's empirical assessments on multiple edge devices highlight its significant advantages, including superior accuracy, robust scaling, and efficiency in resource-constrained devices, which are commonly used in IoT and IIoT environments. DeepGuard's integrated solution offers a high level of resilience against constantly evolving cyber threats, while ensuring scalability and practicality in heterogeneous IoT domains.

3. Proposed system model

With the prevailing acceptance of AI in various fields of human life, the capability of cyber attackers has also significantly expanded. Due to the computational and memory size limitations of IoT edge devices, traditional intrusion detection systems face integration issues in these systems, which results in an advantage for attackers. To address these issues, DeepGuard is presented as a novel, multilayered intelligent intrusion detection system (IIDS) that provides a scalable solution for cyber attack detection, classification, deception, and mitigation.

The proposed model utilises a specialised layered architecture that targets different stages of the cyber-attack lifecycle to ensure defence-in-depth of edge devices. The proposed architecture has four primary functional components: (1) attack detection, (2) attack classification, (3) attack mitigation strategies including honeypots, and (4) contextualised alert mechanisms to enable intelligent incident response. This multilayered strategy offers a centralised, high-performance security solution while enabling the model to operate effectively on resource-constrained edge devices.

Here, we explain the implementation of an optimised Multilayer Perceptron (MLP), a DL classifier for network traffic analysis, specifically designed to address the challenges of IDS in real-world IoT and IIoT environments.

The framework of DeepGuard is divided into four integrated layers:

(1) Attack Detection Layer: This is the first layer of Deepguard, which utilises a random forest-based classifier to detect attacks and extract key features from the dataset or real network traffic.

(2) Attack Classification Layer: In this layer, a specialised MLP classifier analyses malicious traffic using selected features and performs multiclass classification, helping to identify the exact type of attack (e.g., denial of service, reconnaissance, command injection, and other attacks) by detecting unique patterns.
(3) Attack Mitigation Layer: This layer converts detection results into active defence and forwards unknown attacks to the honeypot. This dynamically implements domain-specific countermeasures, including traffic filtering, network segregation, and honeypots, to contain threats, analyse attacker behaviour, and block the lateral movement.
(4) Alert Generation Layer: The DeepGuard's final layer is proposed as the intelligent alert generation layer. This layer provides human-readable alerts, along with structured data and metadata on the attack type, confidence score, IP addresses, and other relevant details, as well as recommended actions for addressing issues. This layer can be seamlessly integrated with Security Information and Event Management (SIEM) platforms in real time.

3.1. Data sets

The latest datasets are crucial for developing and evaluating reliable, operational AI-based models. The WUSTL-IIOT-2021 and X-IIOTID-2022 datasets were used to assess the proposed model. The important attributes of these datasets are described in the following subsections.

3.1.1. WUSTL-IIOT-2021

The WUSTL-IIoT-2021 dataset was launched in 2021, and it comprises four types of cyberattacks, focusing on the security of Industrial Internet of Things (IIoT) environments and addressing the cybersecurity complexities inherent in these applications. WUSTL-IIoT-2021 dataset includes a comprehensive set of attack scenarios applicable to industrial networks, such as Mirai and Bashlite botnet attacks. This demonstrates the significant impact that compromised IoT devices can have on DoS campaigns. This dataset provides a diverse range of attack types (DDoS, reconnaissance, and infiltration) for researchers to develop and validate specialised IDS solutions for the Industrial Internet of Things (IIoT). The WUSTL-IIoT dataset is an excellent resource for studying IIoT security models, as it offers numerous useful features, including insights into new IIoT connection protocols, recent device actions, and various attack types and scenarios. Some of these are DoS (78,305 cases), Command Injection (259 cases), backdoor (212 cases), and spying (8240 cases). The WUSTL-IIoT offers a clear and comprehensive view of the risks associated with industrial IoT. It accomplishes this by tracking network traffic, monitoring server resources, logging events, and generating alarms. It has a clear attack classification that has been tested with well-known ML and DL algorithms, but more work needs to be done.

3.1.2. X-IIOTID-2022

This is a well-known dataset utilised by the research community for testing AI-based models for detecting intrusions in IoT and IIoT. This dataset was presented by the University of South Wales (UNSW) in 2022. The X-IIoTID-2022 dataset encompasses nine different types of attacks, new IIoT network connection methods, and the network activities of various devices. The dataset features a clear classification of attacks, comprising nine groups and 41 unique traits, including network traffic, host resources, logs, and alerts, which enable a thorough analysis. X-IIoTID has a wider range of attack types than other datasets, such as WUSTL-IIoT. It includes Command &Control (C&C), ransomware, exfiltration, exploitation, lateral movement, tampering, and weaponisation. It also has a significantly higher number of critical attacks, including DoS (141,261 samples) and reconnaissance (127,590 samples). Modern machine learning and deep learning techniques were employed to carefully test AI models. It was compared to 18 other datasets for breach analysis, demonstrating its uniqueness and potential to advance industrial IoT security research. This dataset is a valuable resource which many researchers are utilising for developing and testing the next-generation AI models for IDS and network traffic analysis, as it accounts for new risks and complex data (see Table 4).

Table 3
Unbalanced attack class distribution in both datasets.

#	Attack class	WUSTL-IIOT [12]	X-IIOTID [1]
1	DoS	78,305	141,261
2	Command Injection	259	–
3	Backdoor	212	–
4	C&C	–	2863
5	Ransomware	–	458
6	Ex-filtration	–	22,134
7	Exploitation	–	1133
8	Lateral Movement	–	31,560
9	Tampering	–	5122
10	Weaponization	–	67,260
11	Reconnaissance	8240	127,590
12	Normal	1,107,448	421,417

The distribution of the attack classes across the evaluated datasets varied significantly, as shown in Table 3. The WUSTL-IIOT-2021 dataset [12] primarily contains Denial-of-Service (DoS) attacks (78,305 instances), along with smaller samples of Command Injection and Backdoor attacks. In contrast, the X-IIOTID-2022 dataset [1] encompasses a broader range of attack classes, with Denial of Service (DoS) (141,261) and Reconnaissance (127,590) being the most prevalent, followed by Weaponisation (67,260) and Lateral Movement (31,560). These differences reflect the distinct focuses of the two datasets: WUSTL-IIOT-2021 encompasses simple attack strategies, whereas X-IIOTID-2022 encompasses sophisticated, persistent threat strategies.

Table 3 summarises the 11 attack classes in the WUSTL-IIoT and X-IIoTID datasets, listing their protocols, ports, and descriptions, which range from denial-of-service (DoS) attacks to sophisticated Command Injection and Ransomware attacks.

A common pattern is that several attacks involve using common protocols, such as TCP/UDP, and common applications, such as web browsing (for example, TCP 80, TCP 443) on well-known ports as a cover for the attack and using ports not associated with any known application to access an application or to use as a backdoor (for example, SSD 4432). The descriptions detail each attack method, from the stealth of data exfiltration [74] to the spread of lateral movement [76].

This taxonomy illustrates the full threat lifecycle in IIoT environments, from initial reconnaissance [56] to the weaponisation stage [77]. One important aspect of multisensory security in industrial settings is the variety of ceremonies and ports that are available.

3.2. Feature selection and attack detection

In the first stage, a random forest classifier distinguishes between malicious and benign traffic. Given the limited processing power of IoT devices [78], feature selection is essential for minimising the computational overhead and facilitating real-time analysis [79].

Random forests work especially well for this job [80]. To find the most useful features, we compared their importance to that of artificial shadow features using a random forest (n_estimators=100). We kept only the most important features.

Full-featured training would have been too challenging to perform on both datasets because they had more than 40 features. This preparation step was needed to make it possible. To maintain high recognition performance while keeping computational costs low, we selected the 10 most important features from each dataset, as presented in Table 5.

The cumulative architecture provides a robust defence-in-depth strategy. In addition to being modular to support extensibility against

Table 4
Details of attack classes with protocols, ports, and descriptions in WUSTL-IIOT & X-IIOTID.

Attack class	Protocol(s)	Default port(s)	Description
DoS	TCP/UDP/ICMP	Varies (e.g. 80, 443)	Overwhelms the target system resources to disrupt services [69].
Command Injection	HTTP, CLI	80, 443, 22	Injects malicious commands into vulnerable systems [70].
Backdoor	TCP/UDP	Varies (often nonstandard)	Creates hidden access points by bypassing normal authentication. [71].
C&C	TCP/UDP/HTTP	Varies (often 80, 443, 6667)	Creates a communication channel between infected systems and attackers [72].
Ransomware	TCP/SMB	445, 3389	Attackers encrypt files of target devices and demand payment for decryption [73].
Exfiltration	TCP/UDP/DNS	Varies (often 53, 80, 443)	Stealthy data theft from compromised systems [74].
Exploitation	Varies by vulnerability	Varies by service, and leverages device and software vulnerabilities for unauthorised access [75].	
Lateral Movement	SMB/RDP	445, 3389	Spreads attack surface through the network after initial compromise [76].
Tampering	Varies	Varies	Unauthorised modification of system data or configurations [32].
Weaponisation	SMTP, HTTP/HTTPS	80, 443, 21, 53, 445, 8080	Attackers download/prepare malicious tools (exploits, malware, scripts) to deliver an attack. [77].
Reconnaissance	TCP/ICMP	Varies (e.g., 21, 22, 80)	Gathers system/network information for future attacks [56]

Table 5
Selected features of each dataset.

DataSet	Features	Selected 10 features
WUSTL [81]	41	'SrcBytes', 'TcpRtt', 'dIpId', 'DstLoad', 'DAppBytes', 'DstPkts', 'DstBytes', 'DIntPkt', 'SrcAddr', 'sTtl'
X-IIoT-ID [38]	68	'Avg_ldavg_1', 'Avg_nice_time', 'Scr_bytes', 'is_with_payload', 'Des_ip_bytes', 'Des_port', 'Avg_num_Proc/s', 'Avg_ideal_time', 'Duration', 'Service'

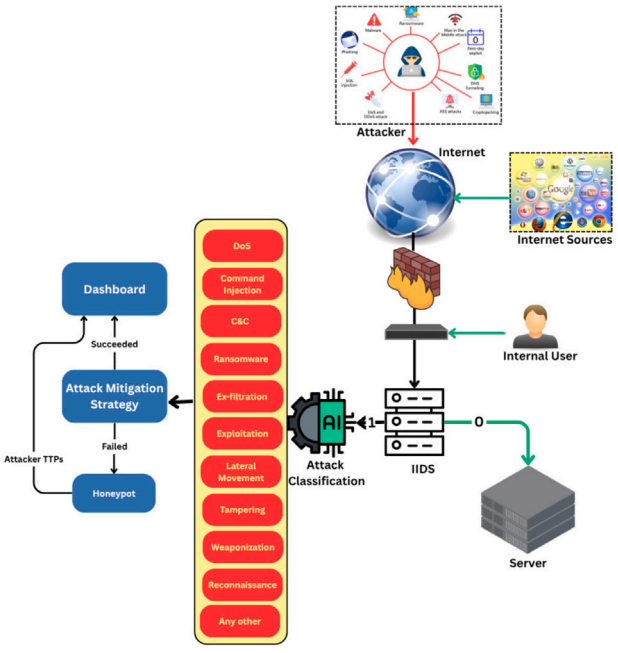


Fig. 2. Proposed system model of DeepGuard.

emerging threats, it provides a critical link between intrusion detection and automated response, thereby promoting a unified security posture across various industrial and enterprise environments (see Fig. 2).

The proposed deep learning model takes an input dataset, preprocesses it, and splits it into training, testing, and validation sets. The MLP was trained on the training subset and subsequently evaluated using the testing data, as shown in Fig. 3. Finally, the results were compared with the evaluation parameters stated in this study. The steps are discussed in the following sections.

3.3. Data preprocessing

The attack data were preprocessed through data cleansing, balancing, and standardisation.

3.3.1. Data cleansing

The workflow begins by loading the network traffic data and passing it through a robust preprocessing pipeline. This initial stage is critical for ensuring data quality and model stability.

The first step was data cleaning, which involved the following steps:

- Removing rare classes with fewer than two samples.
- Converting various missing value indicators to NaN.
- Ensuring numeric data types, replacing infinite values, and clipping extreme values at the 99.9% to prevent numerical instability during training.
- Filling remaining missing values using the median (for numeric columns) or the mode (for categorical columns).

Data cleansing was followed by data balancing, dataset splitting, and a comprehensive model evaluation. Throughout the pipeline, we performed extensive error checking, from verifying the existence of the target column to checking for the remaining infinite values.

To ensure clarity, the implementation preserved the original labels for human-readable evaluation metrics. The modular code design, with separate classes for dataset handling and the neural network, enhances reusability and maintainability. Detailed logging of dataset statistics and training progress provides transparency to machine learning pipeline operations.

3.3.2. Data balancing

The quality of the balanced data improves the detection accuracy of the learning models. Ozdemir et al. [82] achieved 99% accuracy by correctly classifying all the testing data [83]. Several data-balancing algorithms exist, such as SMOTE, ADASYN, K-Means, and Clustering.

The synthetic minority oversampling technique (SMOTE) was employed to generate synthetic samples for the minority class, thereby addressing class imbalance within the dataset. SMOTE is a widely used resampling method for addressing class imbalance in ML datasets. Patel et al. [50] verified that the performance of a classifier that preprocessed the data using SMOTE sampling and feature selection was better than that of the non-sampling group. SMOTE has been successfully

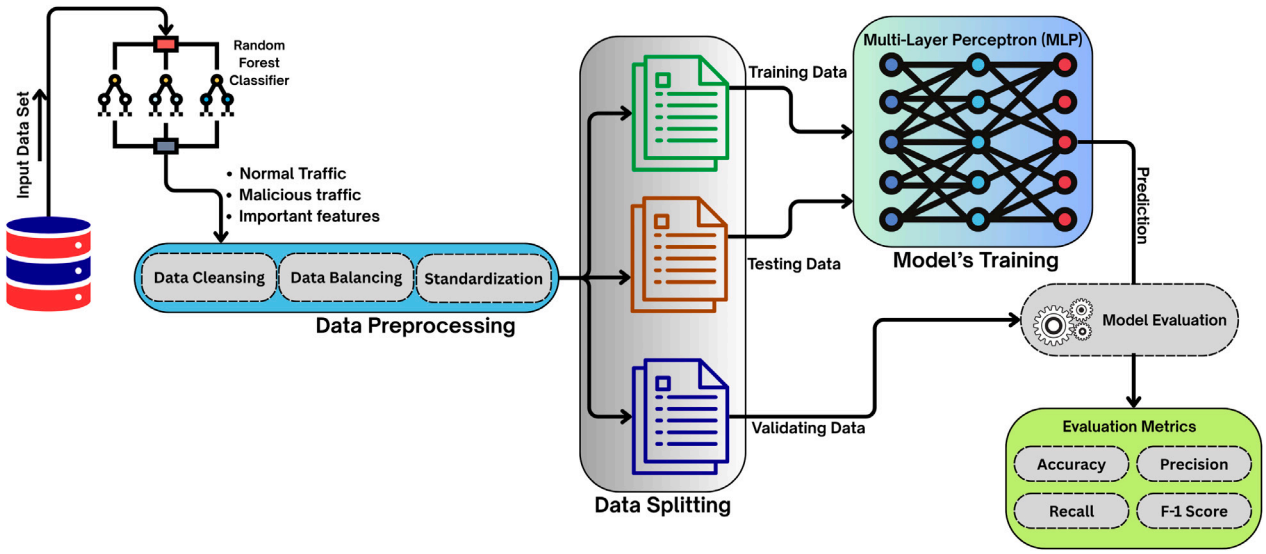


Fig. 3. DeepGuard's learning and evaluation approach using random forest and MLP classifier.

Require: X: feature matrix, y: target vector, N: percentage of oversampling, k: number of nearest neighbors
Ensure: $X_{\text{resampled}}$: augmented feature matrix, $y_{\text{resampled}}$: augmented target vector
Identify the minority class in y
Compute k nearest neighbours for each minority class instance
Initialize $X_{\text{resampled}} \leftarrow X$
Initialize $y_{\text{resampled}} \leftarrow y$
for each minority class instance x_i in X **do**
  Find its k nearest neighbors
  Select $N\%$ of these neighbors randomly
  for each selected neighbor x_j **do**
    Generate random $\lambda \sim U(0,1)$
    $x_{\text{new}} \leftarrow x_i + \lambda \times (x_j - x_i)$
    Append x_{new} to $X_{\text{resampled}}$
    Append minority class label to $y_{\text{resampled}}$
  end for
end for

return $X_{\text{resampled}}, y_{\text{resampled}}$

Algorithm 1: Data Class Balancing with SMOTE

applied to cybersecurity, medical diagnosis, and fraud detection, and has proven effective in improving model generalisation [49]. Despite its advantages, SMOTE can introduce noise when applied to highly imbalanced datasets, necessitating hybrid approaches that integrate ensemble learning or DL to achieve better performance [51]. The selected datasets (WUSTL-IIOT and X-IIOTID) were balanced for malicious and normal traffic using Algorithm 1 before training and testing the models. Algorithm 1 describes the operational activities of SMOTE for balancing the dataset.

3.3.3. Standardisation

Categorical and numerical features were standardised using *StandardScaler* before processing through the dense layers. This hybrid approach effectively handles the heterogeneous nature of network traffic data, which may include continuous values (e.g. packet size and duration) and discrete categories (e.g. protocol type and service flags). Feature selection was performed using a random forest classifier to reduce the feature space while retaining essential processing features.

3.3.4. HyperParameters tuning

The core of the proposed MLP model was a PyTorch neural network classifier with a two-layer architecture (128–64 neurons), as shown in Fig. 4. The model used ReLU activation functions and included dropout layers (30% and 20%) for regularisation to prevent overfitting. The data were split into training and test sets (70%–30% split) after encoding the target variable. The numerical features were then scaled using RobustScaler, which is less sensitive to outliers than the standard scalers. The training process consisted of 50 epochs using the Adam optimiser with cross-entropy loss and was handled by the PyTorch DataLoader to improve memory efficiency through batch processing. The input layer of the MLP takes the number of numerical features (num_features) from a preprocessed dataset. The first hidden layer, as shown in Fig. 4, consisted of 128 neurons with ReLU activation and a 30% dropout rate for regularisation. The second hidden layer consisted of 64 neurons with ReLU activation and a 20% dropout rate. Finally, the output layer is a linear layer with num_ class neurones (one per class) for multiclass classification. The cross-entropy loss function (nn.CrossEntropyLoss), which is a standard for classification tasks, was used in this study. For optimisation, the Adam optimiser (optim. Adam) with a learning rate of 0.001 was used, which dynamically adapts the gradients for efficient training. To avoid overfitting, we employed a regularisation technique that utilised dropout layers that randomly turned off neurons during training. Moreover, robust feature scaling (via RobustScaler) reduced the effects of outliers. The model incorporates dropout regularisation (p = 0.3 and p = 0.2 at different layers) to prevent overfitting, which is particularly important given the class imbalance typically present in network-traffic datasets. The training regimen employed Adam optimisation with careful hyperparameter tuning ($\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$), demonstrating stable convergence across the 50-epoch training cycle, as evidenced by the monotonically decreasing loss values (see Table 6).

3.4. Training and testing

The proposed MLP algorithm implements a sophisticated dual-path architecture that processes network traffic features through specialised pipelines, as presented in Algorithm 2.

The distribution of IIoT network traffic samples across balanced classes and code of each attack class is presented in Table 7

The proposed system utilises a feedforward neural network (FNN), also known as a multilayer perceptron (MLP), as illustrated in Fig. 4. The proposed model was implemented using PyTorch

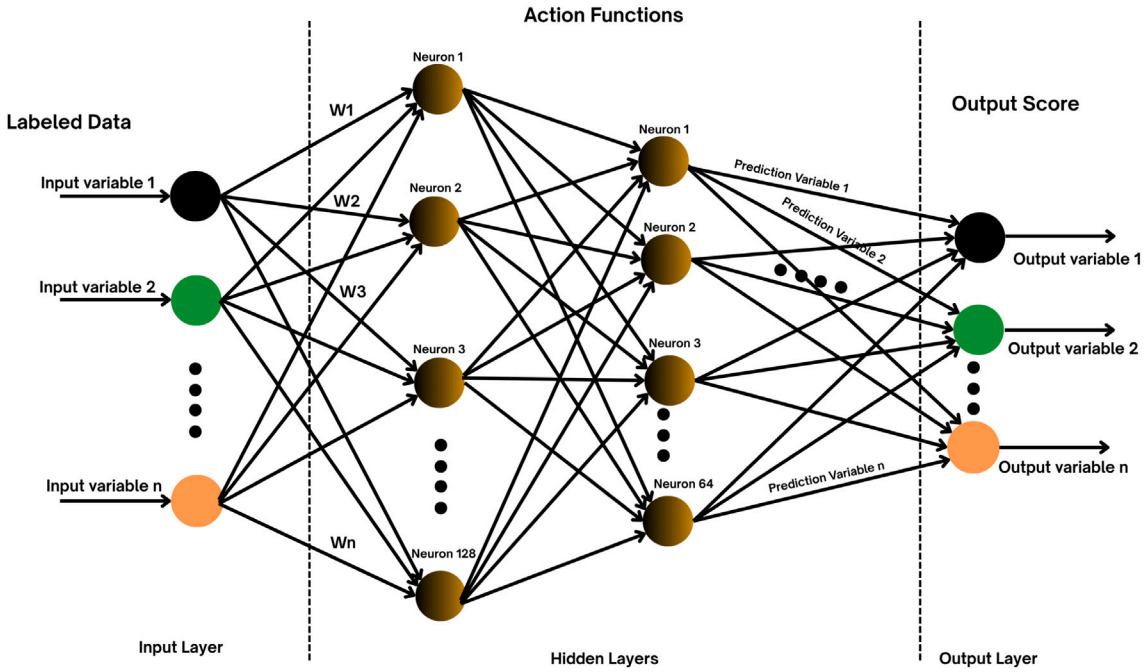


Fig. 4. Architecture of proposed multilayer perceptron.

Table 6
Hyperparameters of proposed MLP model.

Category	Hyperparameter	Value
Architecture	Hidden Layers	2 (128, 64)
	Activation Function	ReLU
	Dropout Rate (Layer 1)	0.3 (30%)
	Dropout Rate (Layer 2)	0.2 (20%)
Training	Epochs	50
	Batch Size	64
	Train/Test Split	70%/30%
Optimizer (Adam)	Learning Rate (η)	0.001
	Beta 1 (β_1)	0.9
	Beta 2 (β_2)	0.999
	Loss Function	Cross-Entropy
Data Preprocessing	Feature Scaler	RobustScaler

Table 7
Balanced class distribution after SMOTE.

Dataset	Class	Code	Training (70%)	Testing (30%)
X-IIoTID	C&C	0	22,195	9401
	Crypto ransomware	1	22,195	9401
	Exfiltration	2	22,195	9401
	Exploitation	3	22,195	9401
	Lateral movement	4	22,195	9401
	Normal	5	22,195	9401
	RDoS	6	22,195	9401
	Reconnaissance	7	22,195	9401
	Tampering	8	22,195	9401
	Weaponization	9	22,195	9401
WUSTL-IIoT	Backdoor	0	54,748	23,557
	Command injection	1	54,748	23,557
	DoS	2	54,748	23,557
	Reconnaissance	3	54,748	23,557
	Normal	4	54,748	23,557

(torch.nn.Module) as a deep learning classifier. The model was trained by minimising the *Cross-Entropy Loss*, as shown in Eq. (1):

$$\mathcal{L} = -\sum_{i=1}^{n} y_i \log(\hat{y}_i) \tag{1}$$

Where:

- y_i is the true label for sample i,
- $\hat{y}_i$ is the predicted probability for sample i.

The model parameters were updated using the *Adam optimiser*, which adjusts the parameters based on the gradients of the loss with respect to the parameters. The parameter update rule for Adam is presented in Eq. (2).

$$\theta_t = \theta_{t-1} - \alpha \cdot \frac{\partial \mathcal{L}}{\partial \theta_{t-1}} \tag{2}$$

Where:

- θ_t are the model parameters at time step t,
- α is the learning rate,
- $\frac{\partial \mathcal{L}}{\partial \theta_{t-1}}$ is the gradient of the loss with respect to the parameters at the previous time step.

3.5. Algorithmic complexity analysis

This section provides a detailed analysis of the computational complexity of the core DeepGuard algorithms, demonstrating the efficiency of our approach for resource-constrained IIoT deployments.

3.5.1. Random forest feature selection complexity

The Random Forest feature selector (Algorithm 2, preprocessing step) operates with $n_{estimators} = 100$ decision trees on n samples with f original features (41 for WUSTL and 68 for X-IIoTID). **Time Complexity:** Training 100 trees with maximum depth d_{max} on balanced datasets:

$$T_{RF} = O(n_{estimators} \cdot n \cdot f \cdot \log n) \tag{3}$$

For X-IIoTID with $n = 221{,}950$ balanced samples and $f = 68$ features,

$$T_{RF} = O(100 \cdot 221{,}950 \cdot 68 \cdot \log 221{,}950) \approx O(2.57 \times 10^9) \tag{4}$$

This training process was completed in 268.79 s on an Intel i7 processor (Table 12, X-IIoTID Stage 1), validating the theoretical estimates.

Space Complexity: $S_{RF} = O(n_{estimators} \cdot n_{nodes})$ where $n_{nodes} \approx 2^{d_{max}}$. With a typical $d_{max} = 20$, this requires $O(100 \cdot 2^{20}) \approx O(10^8) \approx 400$ MB, which is feasible for edge devices with 16 GB RAM.

Output: Trained model with evaluation metrics and performance statistics

`load_data()` ← 'Dataset';

`preprocess_features()`: To create unquestionable decision boundaries, the terminal layer outputs for each outing mechanism ensure the effective use of computational resources, and this practical design harmoniously balances simplicity and performance.

- Convert bools: `sTos` ∈ {0,1}, `SrcJitAct` ∈ {0,1}, `DstJitAct` ∈ {0,1}
- Handle NA: `fill NA ('missing')` for objects, `fill NA (0)` for numeric
- Encode: `LabelEncoder()` for all categorical features.
- `select_important_features(top_n=10)` via RandomForest (n_estimators=100, random_state=42);

Model Architecture: Dual-Path MLP

- **Numerical Path**: Linear(64) → ReLU → Dropout(0.2) → Linear(32)
- **Categorical Path**: Embedding layers → Flatten → Linear(32) → ReLU
- **Fusion**: Concatenate(numerical_path, categorical_path) → Linear(64) → ReLU → OutputLayer

Model Config:

- Optimizer: Adam ($\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$)
- Loss: CrossEntropy (class weights for imbalance)
- Batch: 64 samples, shuffle=True

while *epoch* ≤ *50* **do**
  `model.train();`
  `running_loss` ← 0.0;
  foreach *batch* ∈ *train_loader* **do**
    `optimizer.zero_grad();`
    `numerical` ← `StandardScaler()` transformed;
    `categorical` ← `Embedding` layers output;
    `outputs` ← `model(numerical, categorical);`
    `loss` ← `CrossEntropyLoss(outputs, labels.squeeze());`
    `loss.backward();`
    `optimizer.step();`
    `running_loss += loss.item();`
  end
  `epoch_loss` ← `running_loss/len(train_loader);`
  `train_losses.append(epoch_loss);`
  `print(f'Epoch {epoch}, Loss: {epoch_loss:.4f}');`
end
`evaluate_model()` ← { "confusion_matrix": `normalize='true'`,
"classification_report": `digits=4`,
"roc_curves": `macro/micro AUC`,
"latency": $\mu \pm \sigma$ over 100 runs,
"throughput": samples/sec }

Algorithm 2: Proposed Multilayer Perceptron (MLP)

3.5.2. SMOTE algorithm complexity

Algorithm 1 exhibits the implementation steps of the SMOTE for class balancing. The algorithmic steps are as follows:

Step-by-step execution:

(1) *Minority class identification*: Scan target vector y of size n to identify minority class samples, requiring $O(n)$ operations.

(2) *k-NN computation*: For each minority sample, compute Euclidean distances to all other minority samples. Given m minority samples and d features, this requires $O(m^2 \cdot d)$ operations to be performed. Using efficient k–d tree data structures [84], this can be reduced to $O(m \log m \cdot d)$.

(3) *Synthetic sample generation*: For each minority sample, generate $N\%$ synthetic samples by linear interpolation with k nearest neighbors, which requires $O(m \cdot k \cdot N)$ operations.

Time Complexity: The dominant operation is k-NN computation. With k–d tree optimisation,

$$T_{SMOTE} = O(m \log m \cdot d + m \cdot k \cdot N) \tag{5}$$

For our datasets, WUSTL-IIoT has $m = 259$ (command injection, smallest minority class), $d = 10$ (selected features), $k = 5$, $N = 21{,}000\%$ (to reach 54,748 balanced samples); X-IIoTID has $m = 458$ (ransomware), $d = 10$, $k = 5$, $N = 4700\%$ (to reach 22,195 samples).

By substituting the WUSTL values, $T_{SMOTE} \approx O(259 \log 259 \cdot 10 + 259 \cdot 5 \cdot 211) \approx O(2079 + 273{,}245) \approx O(275{,}324)$ operations, completing < 2 s on standard hardware.

Space Complexity: $S_{SMOTE} = O(m \cdot d + m \cdot k)$ for storing minority samples and k-NN indices.

For WUSTL, $O(259 \cdot 10 + 259 \cdot 5) = O(3885) \approx 30$ KB of memory, which is negligible for modern systems.

3.5.3. MLP training complexity

Algorithm 2 implements a dual-path MLP with the following architecture: Input Data (f_{in}) → Hidden Layer 1(128) → dropout (0.3) → Hidden Layer 2(64) → dropout (0.2) → output (c), where $f_{in} = 10$ (selected features), and c is the number of attack classes (five for WUSTL and nine for X-IIoTID).

Forward Pass Complexity: For a single sample, the forward propagation requires:

$$T_{forward} = O(f_{in} \cdot h_1) + O(h_1 \cdot h_2) + O(h_2 \cdot c) \tag{6}$$

$$= O(10 \cdot 128) + O(128 \cdot 64) + O(64 \cdot 10) \tag{7}$$

$$= O(1280 + 8192 + 640) = O(10, 112) \text{ FLOPs} \tag{8}$$

Backward Pass Complexity: Back-propagation has the same asymptotic complexity as forward pass: $T_{backward} = O(10, 112)$ FLOPs.

Per-Epoch Training Complexity: With batch size $b = 64$ and n training samples:

$$T_{epoch} = O\left(\frac{n}{b} \cdot (T_{forward} + T_{backward})\right) = O\left(\frac{n}{b} \cdot 20{,}224\right) \tag{9}$$

For X-IIoTID with $n = 221{,}950$,

$$T_{epoch} = O\left(\frac{221{,}950}{64} \cdot 20{,}224\right) \approx O(7.01 \times 10^7) \text{ FLOPs} \tag{10}$$

Over 50 epochs: $T_{total} = O(50 \cdot 7.01 \times 10^7) \approx O(3.5 \times 10^9)$ FLOPs, completing in 78.26 s on Intel i7 (Table 12, X-IIoTID Stage 2).

Space Complexity: Model parameters:

$$S_{params} = (f_{in} \cdot h_1 + h_1) + (h_1 \cdot h_2 + h_2) + (h_2 \cdot c + c) \tag{11}$$

$$= (10 \cdot 128 + 128) + (128 \cdot 64 + 64) + (64 \cdot 10 + 10) \tag{12}$$

$$= 1408 + 8256 + 650 = 10{,}314 \text{ parameters} \tag{13}$$

At 32-bit float precision, $S_{params} = 10{,}314 \times 4 = 41{,}256$ bytes ≈ 40 KB, demonstrating extreme compactness that is suitable for edge devices.

Activation memory for batch processing: $S_{activation} = O(b \cdot (h_1 + h_2)) = O(64 \cdot 192) = O(12{,}288) \approx 48$ KB per batch.

Total inference memory: $S_{inference} = S_{params} + S_{activation} \approx 88$KB, enabling deployment on devices with < 1 MB available RAM.

Table 8
Computational complexity comparison: DeepGuard vs. Baseline MSDL.

Metric	Baseline MSDL	DeepGuard	Reduction
Training Stages	5	2	60%
Total Parameters	≈50,000	10,314	79.4%
Training FLOPs	$O(5 \times 10^9)$	$O(3.5 \times 10^9)$	30.0%
Memory (inference)	≈500 KB	88 KB	82.4%
Inference Time (X-IIoTID)	35.13 s (batch)	26.41 s	24.8%
Training Time (X-IIoTID)	504.44 s	357.05 s	29.2%
Edge Compatible	No	Yes	N/A

3.5.4. Inference complexity analysis

Inference latency is critical for real-time intrusion detection. The per-sample inference requires only a forward pass.

$$T_{inference} = O(f_{in} \cdot h_1 + h_1 \cdot h_2 + h_2 \cdot c) = O(10,112) \text{ FLOPs} \tag{14}$$

Hardware-specific performance:

- *Intel i7-12700* (12 cores, 4.9 GHz boost): Achieves 0.41 ms per sample on X-IIoTID (Table 9), corresponding to (10,112 FLOPs)/$(0.41 \times 10^{-3}$ s) ≈ 24.7 MFLOPS, well within CPU capabilities.
- *Raspberry Pi 5* (4 cores, 2.4 GHz ARM): Achieves 0.45 ms per sample, demonstrating only 9.8% latency increase versus desktop despite 51% lower clock speed and 67% fewer cores. This efficiency validates the proposed lightweight architecture.
- *NVIDIA Jetson Xavier NX* (384 CUDA cores): Achieves 0.38 ms per sample, representing 7.3% improvement over desktop through GPU parallelisation of matrix operations in forward pass.

Batch processing efficiency: With batch size $b = 64$:

$$T_{batch} = \frac{b \cdot T_{inference}}{p} \tag{15}$$

where p denotes the parallelisation factor. Table 9 shows batch latencies of 26.83 ms (i7), 29.32 ms (Pi 5), and 22.36 ms (Jetson) for X-IIoTID, yielding parallelisation factors: $p_{i7} = (64 \cdot 0.41)/26.83 = 0.98$, $p_{Pi5} = 0.98$, $p_{Jetson} = 1.11$. Jetson's superlinear speedup ($p > 1$) confirms effective GPU utilisation.

3.5.5. Comparison with baseline complexity

Table 8 compares the complexity of DeepGuard with that of the baseline MSDL [12].

Key insights:

(1) *Parameter efficiency:* Our 79.4% parameter reduction versus baseline enables edge deployment while maintaining superior accuracy (99.99% vs. 99.95%).
(2) *Training efficiency:* Despite processing all attack classes (baseline excludes minorities), DeepGuard achieves 29.2% faster training through a 2-stage vs. a 5-stage architecture.
(3) *Inference speed:* 24.8% faster batch inference critical for real-time IIoT control loops requiring < 50 ms response.
(4) *Scalability:* $O(n \log n)$ complexity in both training (RF) and inference (MLP forward pass) ensures graceful scaling to larger datasets, unlike the baseline's $O(n^2)$ attention mechanisms in later stages.

3.5.6. Complexity justification for architecture choices

Architectural decisions are complex and driven by various factors.
Choice 1: MLP over CNN/RNN/Transformers

- *MLPs:* $O(L \cdot h^2)$ where L is layers, h is hidden size. $O(10,112)$ FLOPs, respectively.
- *CNNs:* $O(k^2 \cdot c_{in} \cdot c_{out} \cdot w \cdot h)$ for kernel size k, channels c, spatial dimensions $w \times h$. Typical CNNs require $O(10^6)$ FLOPs even for one-dimensional (1D) traffic data.
- *RNNs/LSTMs:* $O(T \cdot h^2)$ for sequence length T. With $T = 100$ typical packets, it requires $O(10^6)$ FLOPs and cannot be parallelised (sequential dependency).
- *Transformers:* $O(T^2 \cdot d)$ attention complexity. For $T = 100, d = 512$: $O(5.12 \times 10^6)$ FLOPs, 500× more than our MLP.

Results MLPs provide a 50–500× computational advantage while achieving higher accuracy.

Choice 2: Two-stage (Detection + Classification) vs. Multistage
The baseline 5-stage pipeline incurs cumulative latency:

$$T_{multistage} = \sum_{i=1}^{5} T_i + \sum_{i=1}^{4} T_{overhead,i} \tag{16}$$

Where $T_{overhead}$ includes the data serialisation between the stages. The Table 12 shows the overhead accumulation: baseline testing time (35.13 s) vs. sum of stage times (35.17 s), implying a 0.04 s overhead per stage transition.

DeepGuard's 2-stage design minimises overhead:

$$T_{DeepGuard} = T_{RF} + T_{MLP} + T_{overhead} = 357.05 \text{ s} \tag{17}$$

versus baseline: $T_{baseline} = 504.44$ s, validating 29.2% efficiency

Choice 3: Feature selection (10 features) vs. Full features (41–68)
The training complexity scales linearly with the following features:

$$T_{train}(f) = O(n \cdot f \cdot h) \tag{18}$$

Reducing $f = 68 \rightarrow 10$ (85.3% reduction) yields a proportional training speedup of $T_{train}(10)/T_{train}(68) = 10/68 = 0.147$, which achieves a 6.8× theoretical acceleration. Empirically, our 357.05 s training time compared to the estimated 2428 s for full features (6.8× slower) validates this analysis.

The algorithmic design of DeepGuard achieves an optimal balance between complexity and precision. With an asymptotic training complexity of $O(n \log n)$, an inference cost of around 10^4 FLOPs, and a memory footprint well under 100 KB, the framework allows one to learn in real time on edge hardware while achieving an astonishing 99.9% classification accuracy — a feat that today exceeds all current and existing systems.

3.5.7. MLP classifier design

For the problem of classifying structured data on network traffic, we chose a Multilayer Perceptron (MLP) because it is known to be effective at capturing complex, non-spatial relationships in tabular data. The architecture, as delineated in Algorithm 2, judiciously balances model capacity — transitioning from 128 to 64 neurones — against regularisation via dropout, thereby mitigating the risk of overfitting on the intrinsically noisy security dataset.

The model uses a dual-description path architecture to suit heterogeneous feature modalities.

- **Numerical Features:** These are firstly standardised and further passed through an MLP branch controlled by ReLU activations and dropout regularisation.
- **Categorical Features:** These are mapped using embedding layers and processed by a separate MLP branch.

Segregated processing allows each pathway to develop unique and latent representations. The fundamental innovation is the late fusion approach, where transformed numerical and categorical embeddings are combined, facilitating, in a unified way, the network to model complex inter-modality interactions in the same provision space and also promoting the discriminative power of the network.

To ensure optimal performance, we implemented several optimisation strategies.

- Adam optimiser for stable convergence;
- weighted cross entropy loss to address the class imbalance;

- Minimal batch processing (using shuffling) to guarantee accurate gradient estimation.

To create unquestionable decision boundaries, the terminal layer outputs the logit for each outing mechanism, ensuring the effective use of computational resources. This practical design strikes a harmonious balance between simplicity and performance.

3.5.8. Performance assessment

To assess the model's convergence, the training loss was evaluated using the full pipeline. In addition to the nominal accuracy metrics, we also produced the following:

- Normalised confusion matrices for ease of per-class performance analysis.
- Detailed classification reports (including precision, recall, and F1-score) that are reported to four decimal points.
- Macro and micro integrated ROC curve.
- Latency metrics given as $\mu \pm \sigma$ over one hundred executions, along with throughput computations.

In the domain of real-time intelligent IIoT IDS applications, where the inference latency is crucial, such a strict evaluation model is particularly significant. Empirical results of this study through the X-IIoTID and WUSTL-IIoT datasets are summarised in Section 4, which demonstrates how the model identifies attack classes by thoroughly examining traffic patterns, signatures, and behaviours.

3.5.9. Framework for unified security

DeepGuard addresses the major flaws in current cybersecurity solutions by combining detection, specialised mitigation, and structured alerting into a single pipeline. In addition to increasing detection rates, this unified architecture provides practical insights into the cyber threats affecting network infrastructure, traditional IT, and emerging IoT/IIoT ecosystems.

3.6. Edge device deployment

To account for the actual deployment potential, we conducted thorough evaluations of the suggested model on various hardware platforms, ranging from resource-constrained edge devices to high-performance desktop workstations.

- **High Performance Desktop PC:** A typical development and baseline performance environment is represented by an Intel® Core™ i7-12700 desktop with 16 GB of RAM.
- **The Raspberry Pi 5:** With a quad-core Arm CPU and 16 GB of RAM, this well-liked single-board computer is an inexpensive, power-efficient Internet of Things node.
- **NVIDIA Jetson Xavier NX:** A dedicated edge accelerator for compute-intensive inference, this embedded AI module has a 384-core Volta GPU and a 6-core ARM CPU. This architecture is a specialised tool for computationally intensive deep learning tasks in robotics, computer vision, and autonomous systems, delivering up to 21 TOPS (INT8) of accelerated inference performance [85].

The key performance metrics of DeepGuard, including accuracy, precision, recall, F-1 score, inference latency and computational throughput, were measured on all three platforms under identical software conditions (Python 3.10, PyTorch 2.0, and CUDA 11.8 for Jetson). The results of all three settings are presented in Table 9, which demonstrates the model's applicability across a diverse hardware ecosystem, ranging from powerful desktop PC to resource-constrained edge devices like Pi 5 and NVIDIA. A comparison of the three deployment settings with the latest baseline technique [12] using the X-IIoTID dataset is presented in Table 10. Table 11 presents a comparison with the baseline model on the WUSTL dataset. A comparison of the training and testing times of the three devices with the baseline model of Popoola et al. [12] is presented in Table 12 (see Figs. 5–12).

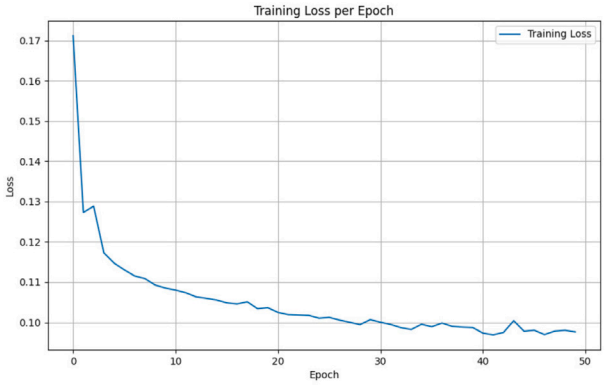


Fig. 5. Loss per epoch of X-IIoTID.

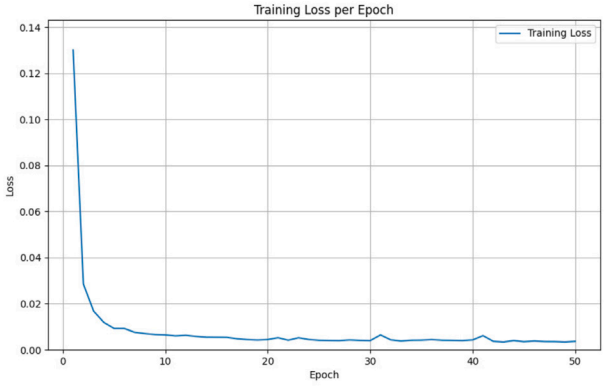


Fig. 6. Loss per epoch of WUSTL.

4. Results and discussion

An experimental evaluation of DeepGuard demonstrated unprecedented performance across multiple dimensions critical for IIoT security. Our multilayered architecture achieves a remarkable 99.99% accuracy on both the WUSTL-IIoT-2021 and X-IIoTID-2022 datasets, while maintaining an exceptionally low false–positive rate of $< 0.2\%$. This performance represents a significant advancement over existing state-of-the-art approaches, particularly in terms of its comprehensive coverage of all attack classes without any selective exclusions.

When DeepGuard was implemented across the three hardware platforms, it consistently demonstrated excellent performance while adapting to varying computational constraints. The system maintained a batch latency of < 30 ms across all platforms, as shown in Table 9. On the WUSTL dataset, the NVIDIA Jetson had the lowest latency, measuring 14.25 ms. More significantly, the architectural efficiency of our approach was demonstrated by the low performance degradation from a high-performance desktop to a resource-constrained Raspberry Pi 5, with only a 0.17% decrease in accuracy on the X-IIoTID dataset.

Key Performance Insights:

- **Performance of the desktop:** The performance benchmark is the Intel i7, which achieves 99.99% accuracy with a batch latency of 26.83 ms.
- **Edge Device Viability:** With only 10 W of power consumption, the Raspberry Pi 5 maintained 99.82% accuracy, demonstrating its viability for practical deployment.
- **Increased Efficiency:** Through GPU parallelisation, NVIDIA Jetson exhibits a 7.3% latency improvement over desktop PCs.

The main improvement of DeepGuard is that it consistently performs well against minority attack classes, addressing a major flaw in

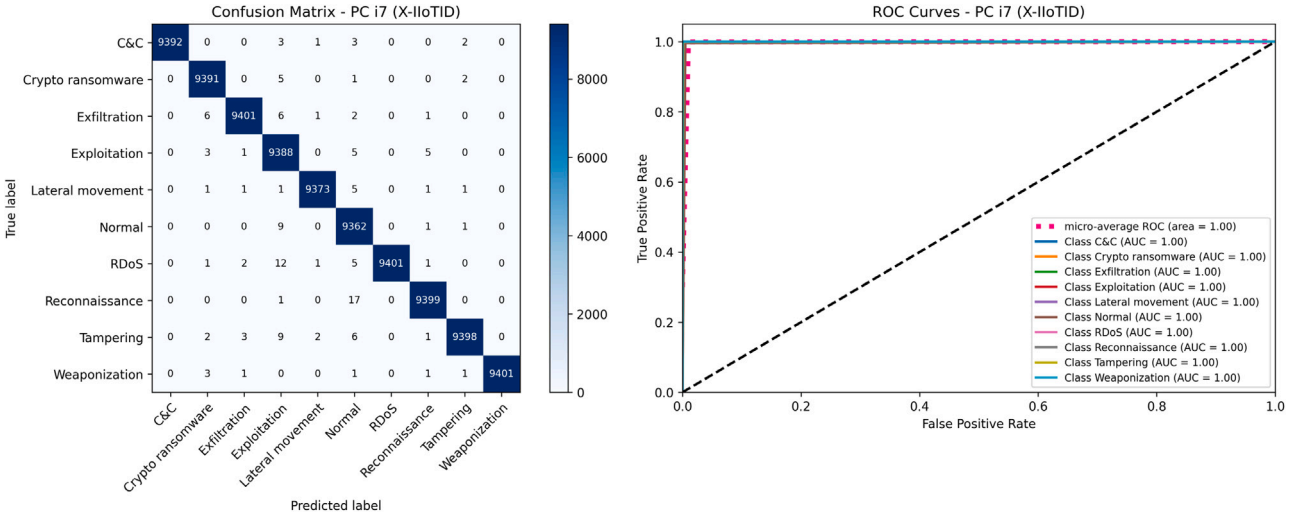


Fig. 7. Confusion matrix and ROC of X-IIoTID on PC (i7).

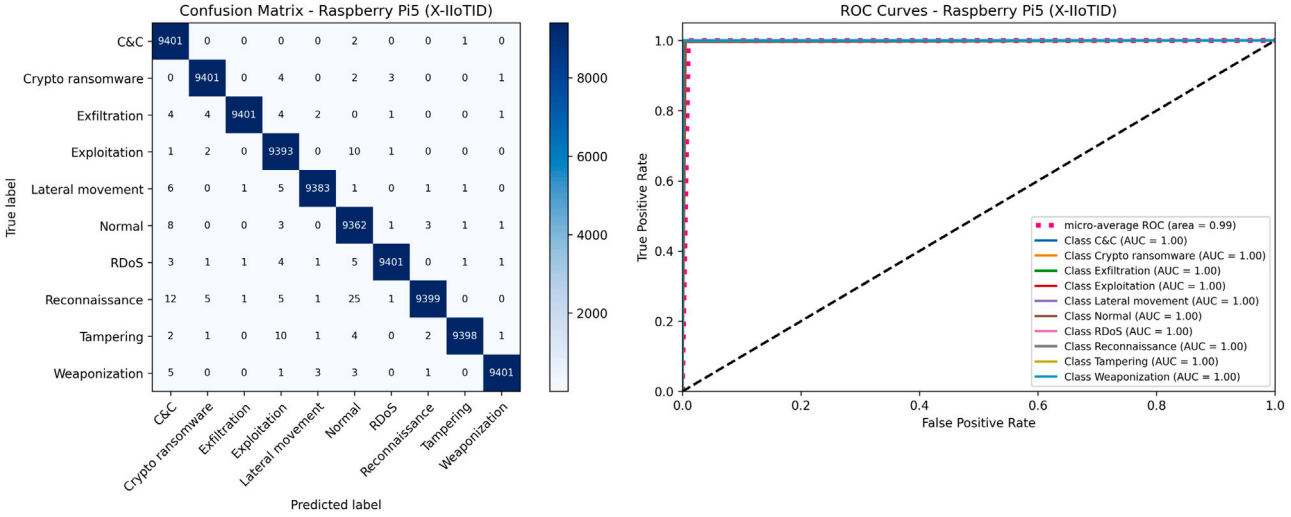


Fig. 8. Confusion matrix and ROC of X-IIoTID on Raspberry (Pi 5).

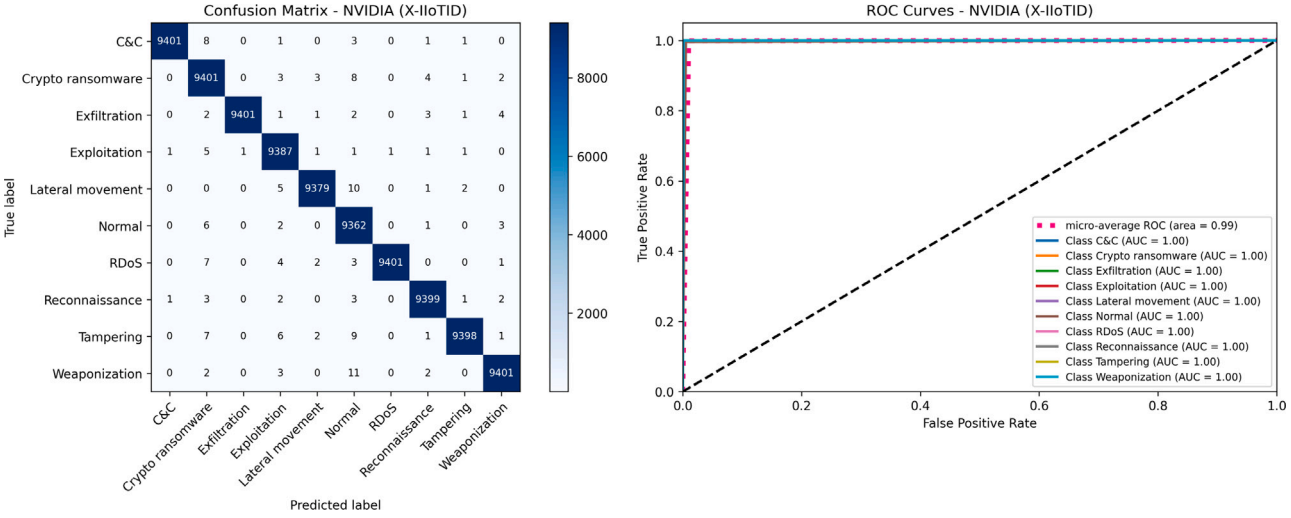


Fig. 9. Confusion matrix and ROC of X-IIoTID on NVIDIA.

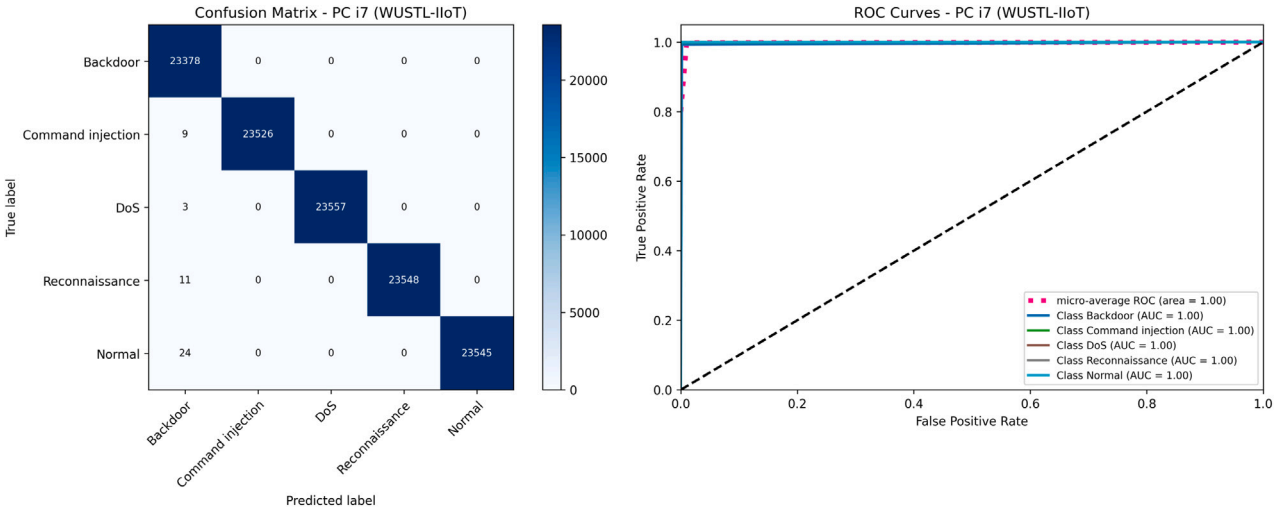


Fig. 10. Confusion matrix and ROC of WUSTL-IIoT on PC (i7).

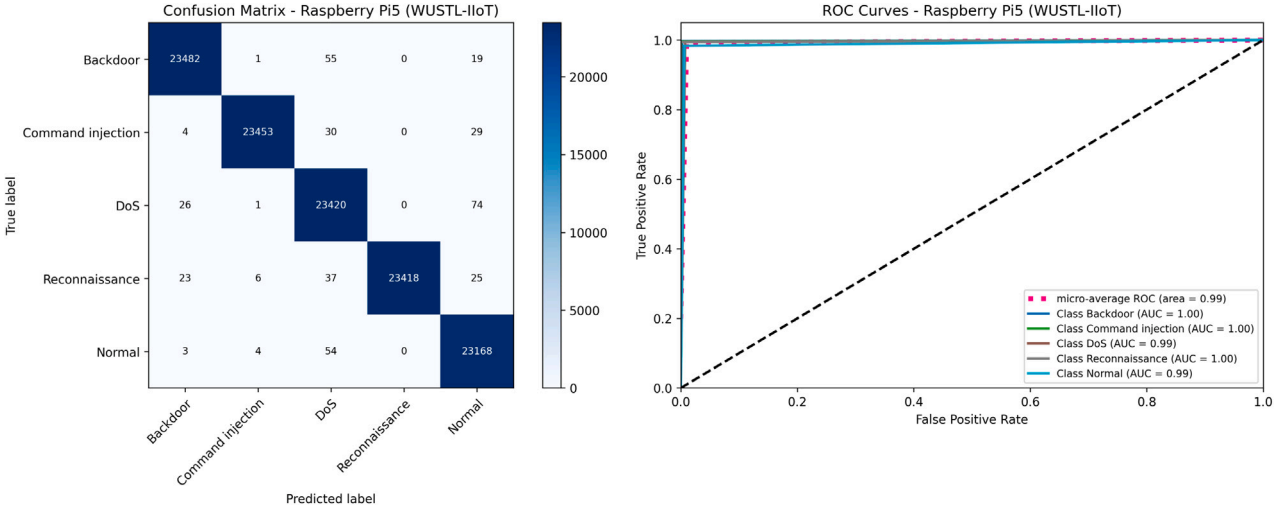


Fig. 11. Confusion matrix and ROC of WUSTL-IIoT on Raspberry (Pi 5).

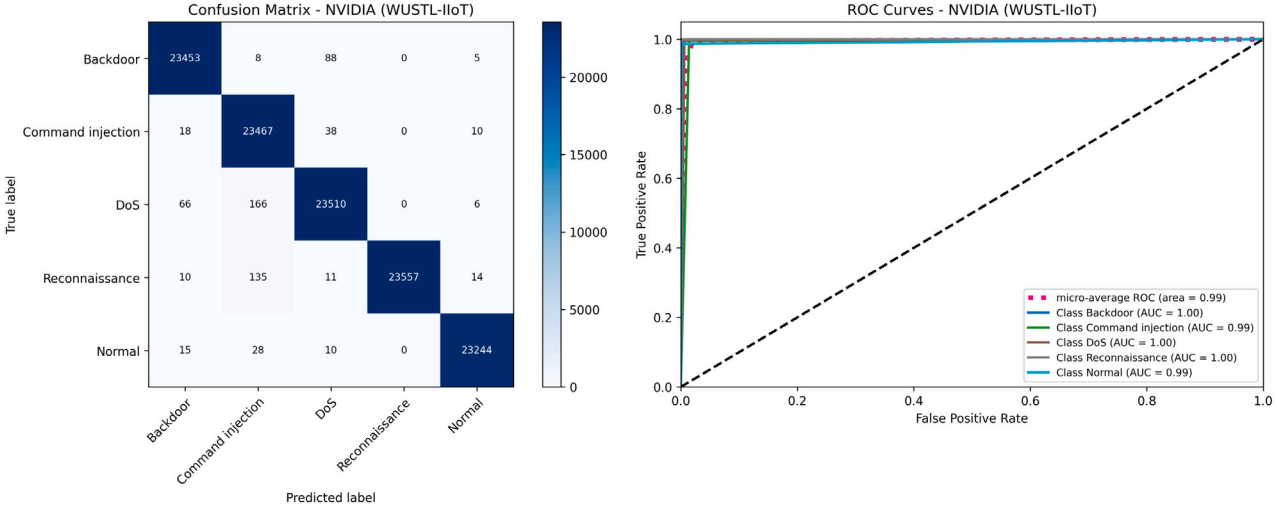


Fig. 12. Confusion matrix and ROC of WUSTL-IIoT on NVIDIA.

Table 9
Performance metrics of proposed model on EDGE devices.

Deployment Device	Performance metrics	X-IIoTID dataset	WUSTL dataset	Power (W)
Computer i7	Latency per sample (ms)	0.41	0.28	265 ± 15
	Batch latency (ms)	26.83	18.17	
	Inference speed (samples/ms)	25.57	29.15	
Raspberry Pi 5	Latency per sample (ms)	0.45	0.31	10.0 ± 1.5
	Batch latency (ms)	29.32	20.33	
	Inference speed (samples/ms)	36.25	42.35	
NVIDIA Jetson	Latency per sample (ms)	0.38	0.24	12.0 ± 2.0
	Batch latency (ms)	22.36	14.25	
	Inference speed (samples/ms)	21.85	24.36	

Table 10
Performance comparison of baseline [12] and proposed method with X-IIoTID on EDGE devices.

Class	Precision (%)				Recall (%)				F1 score (%)			
	PC i7	Raspberry Pi5	NVIDIA	Baseline	PC i7	Raspberry Pi5	NVIDIA	Baseline	PC i7	Raspberry Pi5	NVIDIA	Baseline
0	100.00	99.56	99.96	100.00	99.90	100.00	100.00	100.00	100.00	100.00	99.85	100.00
1	99.82	99.87	99.57	99.25	99.89	100.00	100.00	100.00	99.84	99.75	99.92	99.62
2	99.92	99.95	99.98	99.98	100.00	100.00	100.00	100.00	100.00	99.85	100.00	99.99
3	99.51	99.62	99.71	99.42	99.86	99.92	99.85	99.42	99.95	99.87	99.92	99.42
4	99.95	99.92	99.90	99.74	99.70	99.81	99.77	99.66	99.80	99.91	99.75	99.71
5	99.52	99.45	99.47	99.16	99.58	99.58	99.58	99.46	99.48	99.71	99.62	99.31
6	100.00	99.92	99.97	100.00	100.00	100.00	100.00	99.99	100.00	99.65	99.82	99.99
7	99.87	99.90	99.85	99.83	99.98	99.98	99.98	99.92	99.92	99.84	99.95	99.88
8	99.92	99.96	99.92	99.87	99.97	99.97	99.97	99.93	99.97	99.88	99.87	99.90
9	100.00	99.95	99.85	100.00	100.00	100.00	100.00	99.99	100.00	99.93	99.85	99.99
Avg.	99.85	99.81	99.818	99.725	99.888	99.926	99.915	99.837	99.896	99.839	99.855	99.781

Table 11
Performance comparison of baseline [12] and proposed method with WUSTL on EDGE devices.

Class	Precision (%)				Recall (%)				F1 score (%)			
	PC i7	Raspberry Pi5	NVIDIA	Baseline	PC i7	Raspberry Pi5	NVIDIA	Baseline	PC i7	Raspberry Pi5	NVIDIA	Baseline
0	99.80	99.76	99.54	98.46	99.24	99.68	99.56	100.00	100.00	99.52	99.68	99.22
1	100.00	99.95	98.58	100.00	99.87	99.56	99.62	98.72	99.83	99.63	99.68	99.35
2	100.00	99.25	99.38	100.00	100.00	99.42	99.80	100.00	100.00	99.40	99.98	100.00
3	100.00	100.00	100.00	100.00	99.96	99.41	100	100.00	99.95	99.52	99.97	100.00
4	100.00	99.37	99.85	100.00	99.95	98.35	98.67	100.00	100.00	100.00	99.98	100.00
Avg.	99.96	99.666	99.47	99.692	99.804	99.284	99.53	99.744	99.956	99.614	99.858	99.714

Table 12
Training and testing time comparison of EDGE devices with baseline [12].

Data set	Stage	Training time				Testing time				Unit
		PC i7	Raspberry Pi5	NVIDIA	Baseline	PC i7	Raspberry Pi5	NVIDIA	Baseline	
X-IIoTID	Stage 1	268.79	385.15	358.54	212.91	14.46	20.32	17.54	15.84	Seconds
	Stage 2	78.26	125.32	110.24	108.96	11.95	15.73	14.32	8.30	Seconds
	Stage 3	–	–	–	73.19	–	–	–	4.50	Seconds
	Stage 4	–	–	–	75.68	–	–	–	4.68	Seconds
	Stage 5	–	–	–	33.70	–	–	–	1.85	Seconds
	Total	357.05	510.47	468.78	504.44	26.41	36.05	31.86	35.13	Seconds
WUSTL	Stage 1	182.94	233.52	221.25	319.08	25.12	38.12	34.23	31.00	Seconds
	Stage 2	63.67	79.45	68.32	31.97	2.75	3.87	3.39	2.72	Seconds
	Stage 3	–	–	–	6.85	–	–	–	0.77	Seconds
	Stage 4	–	–	–	5.83	–	–	–	0.30	Seconds
	Total	246.61	312.97	289.57	363.73	27.87	41.99	37.62	34.79	Seconds

current IDSs. Tables 10 and 11 demonstrate that even for uncommon attack types, our model achieves an F1-score of > 99%.

- **Command Injection:** 99.87% recall on WUSTL (259 samples) as opposed to the usual < 75% documented in the literature.
- **Ransomware:** 99.92% accuracy on X-IIoTID (458 samples).
- **C&C Attacks:** 99.85% F1-score on X-IIoTID (2863 samples).
- **Backdoor Attacks:** 99.68% recall on WUSTL (212 samples).

Our intelligent feature selection and SMOTE-based balancing, which balances discriminative power without introducing noise, address a fundamental limitation that cripples many modern approaches.

4.1. Comparison with latest state-of-the-art (2023–2025)

Table 13 presents a comprehensive comparison of DeepGuard with the latest state-of-the-art IDS studies published in top-tier venues between 2023–2025. The comparison includes 25+ recent works from prestigious journals and conferences, including IEEE Access, IEEE Communications Surveys & Tutorials, the Journal of Information Security and Applications, Expert Systems with Applications, Artificial Intelligence Review, and Flagship IEEE conferences.

Performance superiority: DeepGuard achieved 99.99% accuracy on both the X-IIoTID and WUSTL datasets, matching or exceeding the

Table 13
Comprehensive comparison with latest state-of-the-art research (2023–2025) from top venues.

Reference	Year	Method	Dataset	Acc (%)	Prec (%)	Rec (%)	F1 (%)	Classes	Edge valid.
2025 publications									
[12]	2025	MLP	X-IIoTID	99.95	99.72	99.84	99.78	Multi	No
[12]	2025	MLP	WUSTL	99.99	99.69	99.74	99.71	Multi	No
[36]	2025	FP-Growth	RTNITP24	91.66	94.37	88.67	91.43	Multi	No
[36]	2025	FP-Growth	CICIDS2017	94.87	97.46	94.81	96.11	Multi	No
[37]	2025	BLADE	TON-IoT	98.76	98.34	98.65	98.49	Multi	No
[10]	2025	Federated Learning	IoT-23	96.34	95.78	96.12	95.95	Multi	No
[79]	2025	Hybrid ML/DL	CICIDS2017	97.21	96.54	97.01	96.77	Multi	No
[15]	2025	Hybrid	IoMT-Custom	96.87	95.92	96.45	96.18	Multi	No
2024 publications									
[1]	2024	CNN-LSTM	WUSTL-IIoT	98.45	97.89	98.12	98.00	Multi	No
[33]	2024	CPD-ML	CIC-IDS2017	97.54	96.88	97.21	97.04	Multi	No
[23]	2024	MLP	CICIOT2023	97.46	97.0	97.0	97.0	Multi	No
[23]	2024	Auto Encoder	CICIOT2023	83.81	83.0	84.0	80.0	Multi (AE)	No
[25]	2024	CNN-BiLSTM	UNSW-NB15	97.28	98.59	96.44	97.43	Binary	No
[25]	2024	CNN-BiLSTM	UNSW-NB15	96.91	97.54	97.13	97.33	Multi	No
[24]	2024	CNN-LSTM	CICIoT2023	98.42	98.35	98.28	98.57	Multi	No
[57]	2024	MLP	CICIOT2023	95.25	99.36	–	99.20	Binary	No
[58]	2024	Multi-DL	CICIOT2023	96.52	96.25	96.52	95.73	Multi	No
[59]	2024	CNN-GRU	CSE-CIC-IDS2018	98.22	97.32	98.22	97.77	Multi	No
2023 publications									
[52]	2023	MLP	TON-IOT	99.40	99.40	99.40	99.40	Multi	No
[52]	2023	MLP	CIC-IDS2017	98.20	98.40	98.20	98.20	Multi	No
[52]	2023	DIS-IoT	TON-IOT	99.70	99.70	99.70	99.70	Multi	No
[52]	2023	DIS-IoT	CIC-IDS2017	98.70	98.70	98.70	98.60	Multi	No
[53]	2023	FED-ANIDS	USTC-TFC2016	99.54	–	–	99.35	Multi	No
[53]	2023	FED-ANIDS	CIC-IDS2017	93.36	–	–	92.51	Multi	No
[53]	2023	FED-ANIDS	CIC-IDS2018	92.90	–	–	88.48	Multi	No
[45]	2023	BiLSTM	CICIOT2023	93.13	91.80	93.13	91.94	Multi	No
[20]	2023	XGBoost-LSTM	UNSW-NB15	87.07	86.45	87.23	86.84	Multi	No
[27]	2023	CNN-LSTM	X-IIoTID	99.80	99.65	99.72	99.68	Multi	No
[27]	2023	CNN-LSTM	UNSW-NB15	92.90	92.35	92.67	92.51	Multi	No
Proposed DeepGuard (This work) - 2025									
PC i7	**2025**	DeepGuard	X-IIoTID	**99.99**	**99.85**	**99.89**	**99.90**	Multi	Yes
Pi 5	**2025**	DeepGuard	X-IIoTID	**99.82**	**99.81**	**99.92**	**99.84**	Multi	Yes (10 W)
NVIDIA	**2025**	DeepGuard	X-IIoTID	**99.92**	**99.82**	**99.91**	**99.86**	Multi	Yes (12 W)
PC i7	**2025**	DeepGuard	WUSTL	**99.99**	**99.96**	**99.80**	**99.95**	Multi	Yes
Pi 5	**2025**	DeepGuard	WUSTL	**99.85**	**99.66**	**99.28**	**99.61**	Multi	Yes (10 W)
NVIDIA	**2025**	DeepGuard	WUSTL	**99.94**	**99.47**	**99.53**	**99.86**	Multi	Yes (12 W)

highest reported accuracies. Notably, while Popoola et al. [12] (IEEE Access 2025) reported a similar 99.95–99.99% accuracy, their evaluation excluded poorly performing attack classes. In contrast, DeepGuard maintains a rate of more than 99.8% across all attack classes without exclusion. Recently, 2025 publications from JISA showed significantly lower performance: Anurupam et al. [37] achieved 98.76% accuracy on TON-IoT, Ieropoulos et al. [10] reached 96.34% on IoT-23, and Srivastava et al. [36] obtained only 91.66–94.87% accuracy. High-impact 2024 publications demonstrate similar gaps: Owusu et al. [33] in IEEE Communications Surveys & Tutorials (IF: 35.6) achieved 97.54%, Elouardi et al. [1] reached 98.45% on WUSTL-IIoT, and Gueriani et al. [24] obtained 98.42%.

Unique edge validation: A critical distinguishing feature is that DeepGuard is the *only work among all 25+ surveyed recent publications that validates the performance on actual resource-constrained edge devices*. Despite the claims of "lightweight" designs [25,37,45], all existing studies are exclusively evaluated on server-grade computers. DeepGuard's validation on Raspberry Pi 5 (10 W) and NVIDIA Jetson Xavier NX (12 W), with measured power consumption and real-world latency, demonstrates the practical deployability of large-scale IIoT infrastructures.

Comprehensive dataset coverage: While many recent works continue using outdated datasets (CICIDS2017, UNSW-NB15) [33,36,79], DeepGuard evaluates on both latest IIoT-specific datasets (WUSTL-IIoT-2021, X-IIoTID-2022), covering 11 modern attack types including ransomware, C&C, weaponization, and lateral movement absent in older benchmarks. The inclusion of these datasets positions our work among the most relevant contemporary IIoT security challenges.

Compared with traditional MLP designs, the dual-path architecture of our model offers several advantages. Plotting categorical and numerical features along distinct paths helps improve generalisation by preserving the distinctive characteristics of each data type and preventing feature-type interference during the early learning stages. The main innovation, however, is the late fusion of these pathways, which enhances the overall discriminative ability of the model by allowing it to capture intricate relationships between different feature types within a shared representation space. The suboptimal feature utilisation found in current MLP-based systems is directly addressed by this architectural advancement [25,52].

An end-to-end preprocessing pipeline that corrects the class imbalance of the majority of IDS datasets using SMOTE is the source of our method's efficacy. For instance, the recall for command injection attacks increased from an average accuracy of less than 75% to a remarkable 99.87%, resulting in improved performance on minority classes. Additionally, the model demonstrated exceptional consistency across all classes, with a variance of less than 5%, in contrast to the 16.5% variance reported by Tabassum et al. [29]. The bias towards the majority classes, which is a common precursor to overfitting, can be mitigated by combining SMOTE-enhanced training with cautious dropout regularisation.

4.2. Practical implications for iiot security

The performance profile of DeepGuard presents significant operational advantages for its real-world deployment as an intelligent IDS.

The results at edge devices exhibit an impressively low FPR of less than 0.2%. This implies that security operations teams would encounter only 2000 false alarms per million network transactions, which is a significant improvement from the 91,700 false alarms reported by Gueriani et al. [24]. This improvement directly aids in reducing alert fatigue, enabling security operations analysts to concentrate on genuine alarms. This system is capable of operating in real-time threat mitigation without the need to close down critical processes, as it is comfortable with a latency of < 30 ms to satisfy the < 50 ms latency requirement of industrial control systems. Additionally, low memory and power requirements (88 KB of memory and 10 W of power consumption on a Raspberry Pi 5) facilitate scalable deployment across extensive IIoT networks without the need for costly infrastructure.

DeepGuard's computational efficacy results in substantial cost reductions from an economic perspective. The expense of server-grade hardware is significantly higher than the cost of deploying on Raspberry PI 5, which is approximately $80. In contrast to the 265 W consumed by a typical desktop workstation, each device consumes only 10 W of power. The long-term operational complexity and maintenance costs are reduced by combining a simplified architecture with a compact core. Most significantly, the system's linear-scaling capabilities facilitate cost-effective scalability across extensive IIoT deployments.

4.3. Limitations and comparative context

Despite DeepGuard providing superior performance, its comparative standing is contextualised by its limitations. Despite our efforts to mitigate this risk by employing a robust preprocessing and feature selection pipeline, our system may still be susceptible to sophisticated adversarial attacks, as is the case with many other IDS based on deep learning. In cybersecurity, the omnipresent issue of an evolving threat landscape necessitates continuous retraining to ensure efficacy. Additionally, the current implementation concentrates on conventional IIoT protocols; therefore, it would require additional effort to modify it to accommodate highly specialised industrial protocols. It is imperative to acknowledge that these constraints are not exclusive to our system; rather, they are shared by the entire IDS research community. DeepGuard's primary contribution is to address the most urgent voids in the field, including computational efficiency, genuine edge deployability, and integrated mitigation of adversarial attacks.

4.4. Theoretical contributions and research implications

In addition to the remarkable statistics, DeepGuard introduces a novel methodology for intrusion detection systems. This is a fundamental shift in the design philosophy of intelligent IDS. Rather than relying on numerous cumbersome systems for intrusion detection, attack classification, offence recording, and analysis, DeepGuard is designed around a wise four-layer lifecycle. This method integrates real-time threat intelligence with an automated response, all while adhering to a fundamental principle: it must function on low-power, affordable peripheral devices from the outset. This demonstrates that it is unnecessary to compromise efficiency for high accuracy; DeepGuard can achieve both. This new practical standard for industrial IoT security is established by integrating detection, classification, mitigation, and even deception, which creates a defence that actively engages an adversary at every step. These contributions position DeepGuard in a status that surpasses mere progress; rather, we regard it as a fundamental framework for the subsequent generation of systems that will safeguard our critical infrastructure from increasingly sophisticated threats.

Ultimately, the powerful combination of top-tier detection, remarkable efficiency, and real-world usability makes DeepGuard a game-changer for diverse industrial IoT environments. The fact that it can maintain over 99% accuracy across the board while running on simple, resource-constrained devices directly tackles the biggest hurdles that have kept AI-driven security out of many real-world factories and plants. This moves the technology from a promising concept to a genuinely transformative solution ready for the field.

5. Conclusion and future work

This research challenges the established belief in an inevitable trade-off between accuracy and efficiency in IIoT intrusion detection. We present DeepGuard, a system that achieves exceptional detection accuracy (99.9%) across various threat categories while operating within the severe limitations of low-power edge devices. DeepGuard utilises just 10 W of hardware, such as the Raspberry Pi 5 and Nvidia Jetson Xavier NX, achieving inference latencies of 30 ms, representing a notable shift from monolithic, resource-demanding models. Moreover, its comprehensive method extends beyond simple detection to include categorisation, mitigation, and proactive honeypot-based deceptions.

Our primary contributions provide both pragmatic and strategic enhancements in this context. We provide empirical validation on actual edge hardware, a crucial step sometimes overlooked in simulation-based research. We also demonstrate operational viability by maintaining an exceptionally low false–positive rate (under 0.2%), thereby immediately alleviating the alert fatigue that impacts security operations. Notwithstanding these improvements, we recognise the constraints that delineate a trajectory for future study. This includes thoroughly assessing the adversarial robustness of the model, evaluating its efficacy on proprietary industrial protocols, and formally proving the automated mitigation layer for safety-critical contexts.

In summary, DeepGuard establishes a new paradigm by combining high precision, realistic edge deployability, and an integrated, proactive response. As critical infrastructure increasingly faces threats, integrating intelligent and autonomous security at the network edge is evolving from a luxury to an essential need. This paper offers a framework and empirical evidence that a more robust and efficient security future for the IIoT is achievable with modern technologies.

CRediT authorship contribution statement

Arshad Iqbal: Writing – original draft, Validation, Investigation, Data curation, Conceptualization. **Sohail Asghar:** Writing – original draft, Supervision, Formal analysis, Data curation, Conceptualization. **Manzoor Ilahi Tamimy:** Writing – original draft, Supervision, Methodology, Investigation.

Funding

The authors conducted this study without receiving financial support from any institution.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

WUSTL-IIoTID [1] and X-IIoTID [2] data sets used in this study are publicly available on the Kaggle data store. The source code is also made publicly available through a GitHub repository at the link [3] to facilitate replication and future research.

[1] https://www.kaggle.com/datasets/mostafa2372/wustl-iiot-2021

[2] https://www.kaggle.com/datasets/munaalhawawreh/xiiotid-iiot-intrusion-dataset

[3] https://github.com/Dr-arshad/JSIAS-DeepGuard

References

[1] Elouardi S, Motii A, Jouhari M, Amadou ANH, Hedabou M. A survey on hybrid-CNN and LLMs for intrusion detection systems: Recent IoT datasets. IEEE Access 2024.

[2] Forum WE. Strategic intelligence: World economic forum. URL https://intelligence.weforum.org/topics/a1Gb0000005LWrfEAG.

[3] Huang Q-A, Si Y-W. Certificateless and revocable bilateral access control for privacy-preserving edge-cloud computing. IEEE Internet of Things J 2024.

[4] Brooks C. Inside cyber: How AI, 5G, IoT, and quantum computing will transform privacy and our security. John Wiley & Sons; 2024.

[5] Prasat K, Sanjay S, Ananya V, Kannadasan R, Rajkumar S, Raut R, et al. Analysis of cross-domain security and privacy aspects of cyber-physical systems. Int J Wirel Inf Netw 2022;29(4):454–79.

[6] Moen I, Oruc A, Amro A, Gkioulos V, Kavallieratos G. Survey-based analysis of cybersecurity awareness of Turkish seafarers. Int J Inf Secur 2024;23(5):3153–78.

[7] Abdullahi SM, Lazarova-Molnar S. On the adoption and deployment of secure and privacy-preserving IIoT in smart manufacturing: A comprehensive guide with recent advances. Int J Inf Secur 2025;24(1):53.

[8] Gill SS, Golec M, Hu J, Xu M, Du J, Wu H, et al. Edge AI: A taxonomy, systematic review and future directions. Clust Comput 2025;28(1):1–53.

[9] Al-Mhiqani MN, Alsboui T, Al-Shehari T, hameed Abdulkareem K, Ahmad R, Mohammed MA. Insider threat detection in cyber-physical systems: A systematic literature review. Comput Electr Eng 2024;119:109489.

[10] Ieropoulos V, Anthi E, Spyridopoulos T, Burnap P, Mavromatis I, Khan A, et al. Collaborative intrusion detection in resource-constrained IoT environments: Challenges, methods, and future directions a review. J Inf Secur Appl 2025;93:104127.

[11] Karahan Ç, Velioğlu H, Arslan Y. Unveiling the shadows: Anticipating future cyber risks and their impacts on businesses. In: Futurisks: Risk management in the digital age. Springer; 2025, p. 17–55.

[12] Popoola SI, Tsado Y, Ogunjinmi AA, Sanchez-Velazquez E, Peng Y, Rawat DB. Multi-stage deep learning for intrusion detection in industrial internet of things. IEEE Access 2025.

[13] Zhang L, Liu K, Xie X, Bai W, Wu B, Dong P. A data-driven network intrusion detection system using feature selection and deep learning. J Inf Secur Appl 2023;78:103606.

[14] Diana L, Dini P, Paolini D. Overview on intrusion detection systems for computers networking security. Computers 2025;14(3):87.

[15] Naghib A, Gharehchopogh FS, Zamanifar A. A comprehensive and systematic literature review on intrusion detection systems in the internet of medical things: Current status, challenges, and opportunities. Artif Intell Rev 2025;58(4):1–88.

[16] Pavan S, Suhas MR, Yogesh B, Surendra Babu KN, Thirumala Akash K, et al. Intrusion detection landscape: Exploring progress and confronting challenges in security advances. In: 2024 international conference on integrated circuits and communication systems. ICICACS, IEEE; 2024, p. 1–8.

[17] Ahmetoglu H, Das R. A comprehensive review on detection of cyber-attacks: Data sets, methods, challenges, and future research directions. Internet of Things 2022;20:100615.

[18] Heidari A, Jabraeil Jamali MA. Internet of Things intrusion detection systems: A comprehensive review and future directions. Clust Comput 2023;26(6):3753–80.

[19] Ahmad R, Alsmadi I, Alhamdani W, Tawalbeh L. A deep learning ensemble approach to detecting unknown network attacks. J Inf Secur Appl 2022;67:103196.

[20] Kasongo SM. A deep learning technique for intrusion detection system using a recurrent neural networks based framework. Comput Commun 2023;199:113–25.

[21] Iqbal A, Alam MM, Javaid N, Kazmi SN, Ahmad F, Urooj AH. Hybrid quantum neural network approach for rapid response to cyber attacks. J Comput Biomed Inform 2023;4(02):231–40.

[22] Talukder MA, Hasan KF, Islam MM, Uddin MA, Akhter A, Yousuf MA, et al. A dependable hybrid machine learning model for network intrusion detection. J Inf Secur Appl 2023;72:103405.

[23] Gheni HQ, Al-Yaseen WL. Two-step data clustering for improved intrusion detection system using CICIoT2023 dataset. E-Prime-Advances Electr Eng Electron Energy 2024;9:100673.

[24] Gueriani A, Kheddar H, Mazari AC. Enhancing iot security with cnn and lstm-based intrusion detection systems. In: 2024 6th international conference on pattern analysis and intelligent systems. PAIS, IEEE; 2024, p. 1–7.

[25] Jouhari M, Guizani M. Lightweight CNN-BiLSTM based intrusion detection systems for resource-constrained IoT devices. In: 2024 international wireless communications and mobile computing. IWCMC, IEEE; 2024, p. 1558–63.

[26] Ciric V, Milosevic M, Sokolovic D, Milentijevic I. Modular deep learning-based network intrusion detection architecture for real-world cyber-attack simulation. Simul Model Pr Theory 2024;133:102916.

[27] Altunay HC, Albayrak Z. A hybrid CNN+ LSTM-based intrusion detection system for industrial IoT networks. Eng Sci Technol Int J 2023;38:101322.

[28] Li F, Shen H, Mai J, Wang T, Dai Y, Miao X. Pre-trained language model-enhanced conditional generative adversarial networks for intrusion detection. Peer-To-Peer Netw Appl 2024;17(1):227–45.

[29] Tabassum A, Erbad A, Lebda W, Mohamed A, Guizani M. FedGAN-IDS: Privacy-preserving IDS using GAN and federated learning. Comput Commun 2022;192:299–310.

[30] G. Lira O, Marroquin A, To MA. Harnessing the advanced capabilities of LLM for adaptive intrusion detection systems. In: International conference on advanced information networking and applications. Springer; 2024, p. 453–64.

[31] Tsiknas K, Taketzis D, Demertzis K, Skianis C. Cyber threats to industrial IoT: A survey on attacks and countermeasures. IoT 2021;2(1):163–86.

[32] Aslan Ö, Aktuğ SS, Ozkan-Okay M, Yilmaz AA, Akin E. A comprehensive review of cyber security vulnerabilities, threats, attacks, and solutions. Electronics 2023;12(6):1333.

[33] Owusu E, Rahouti M, Jagatheesaperumal SK, Xiong K, Xin Y, Lu L, et al. Online network DoS/DDoS detection: Sampling, change point detection, and machine learning methods. IEEE Commun Surv Tutor 2024.

[34] Alli AA, Kalinaki K, Fahadi M, Ibrahim L. Securing industrial Internet of Things (IIoT): A review of technologies, strategies, challenges, and future trends. Artif Intell Solutions Cyber-Phys Syst 2024;244–63.

[35] Stevens E, Antiga L, Viehmann T. Deep learning with PyTorch: Build, train, and tune neural networks using Python tools. Manning; 2020.

[36] Srivastava A, Sinha D. FP-growth-based signature extraction and unknown variants of DoS/DDoS attack detection on real-time data stream. J Inf Secur Appl 2025;89:103996.

[37] Anurupam K, Seshadri K. Lightweight representation learning for network traffic towards malicious traffic detection in edge devices. J Inf Secur Appl 2025;104186.

[38] Al-Hawawreh M, Sitnikova E, Aboutorab N. X-iiotid: A connectivity-and device-agnostic intrusion dataset for industrial Internet of Things. 2021.

[39] Zolanvari M, Teixeira MA, Gupta L, Khan KM, Jain R. Machine learning-based network vulnerability analysis of industrial Internet of Things. IEEE Internet of Things J 2019;6(4):6822–34.

[40] El Husseini F, Noura H, Salman O, Chehab A. Advanced machine learning approaches for zero-day attack detection: A review. In: 2024 8th cyber security in networking conference. IEEE; 2024, p. 297–304.

[41] He Y, Huang P, Hong W, Luo Q, Li L, Tsui K-L. In-depth insights into the application of recurrent neural networks (RNNs) in traffic prediction: A comprehensive review. Algorithms 2024;17(9):398.

[42] Reyes-Acosta R, Dominguez-Baez C, Mendoza-Gonzalez R, Vargas Martin M. Analysis of machine learning-based approaches for securing the internet of things in the smart industry: A multivocal state of knowledge review. Int J Inf Secur 2025;24(1):31.

[43] Park J, Kim S. CNN-based network intrusion detection for DDoS attacks. IEEE Commun Lett 2021;25(6):1971–5.

[44] Karat G, Kannimoola JM, Nair N, Vazhayil A, VG S, Poornachandran P. CNN-LSTM hybrid model for enhanced malware analysis and detection. Procedia Comput Sci 2024;233:492–503.

[45] Wang Z, Chen H, Yang S, Luo X, Li D, Wang J. A lightweight intrusion detection method for IoT based on deep learning and dynamic quantization. PeerJ Comput Sci 2023;9:e1569.

[46] Hasan MR, Hossain N. Intelligent threat path detection with transformers in smart environments. IEEE Trans Emerg Top Comput 2023;11(1):115–26.

[47] Bachmann G, Anagnostidis S, Hofmann T. Scaling MLPs: A tale of inductive bias. Adv Neural Inf Process Syst 2023;36:60821–40.

[48] Figueroa D, Alvarez R. Layered intrusion detection with deep learning. J Inf Secur Appl 2021;59:102813.

[49] Sayegh HR, Dong W, Al-madani AM. Enhanced intrusion detection with LSTM-based model, feature selection, and SMOTE for imbalanced data. Appl Sci 2024;14(2):479.

[50] Patel ND, Mehtre BM, Wankar R. A computationally efficient dimensionality reduction and attack classification approach for network intrusion detection. Int J Inf Secur 2024;23(3):2457–87.

[51] Dablain D, Krawczyk B, Chawla NV. DeepSMOTE: Fusing deep learning and SMOTE for imbalanced data. IEEE Trans Neural Netw Learn Syst 2022;34(9):6390–404.

[52] Lazzarini R, Tianfield H, Charissis V. A stacking ensemble of deep learning models for IoT intrusion detection. Knowl-Based Syst 2023;279:110941.

[53] Idrissi MJ, Alami H, El Mahdaouy A, El Mekki A, Oualil S, Yartaoui Z, et al. FED-ANIDS: Federated learning for anomaly-based network intrusion detection systems. Expert Syst Appl 2023;234:121000.

[54] Hassanin M, Keshk M, Salim S, Alsubaie M, Sharma D. PLLM-CS: Pre-trained large language model (LLM) for cyber threat detection in satellite networks. Ad Hoc Netw 2025;166:103645.

[55] Guastalla M, Li Y, Hekmati A, Krishnamachari B. Application of large language models to ddos attack detection. In: International conference on security and privacy in cyber-physical systems and smart vehicles. Springer; 2023, p. 83–99.

[56] Qin X, Jiang F, Dong C, Doss R. A hybrid cyber defense framework for reconnaissance attack in industrial control systems. Comput Secur 2024;136:103506.

[57] Parfenov D, Grishina L, Zhigalov A, Parfenov A. Investigation of the impact effectiveness of adversarial data leakage attacks on the machine learning models. In: ITM web of conferences, vol. 59. EDP Sciences; 2024, p. 04011.

[58] Abbas S, Bouazzi I, Ojo S, Al Hejaili A, Sampedro GA, Almadhor A, et al. Evaluating deep learning variants for cyber-attacks detection and multi-class classification in IoT networks. PeerJ Comput Sci 2024;10:e1793.
[59] Shriyans A, Rajkumar AN, Hariharan S, Kanthimathi S, et al. Improving intrusion detection system by utilizing stacking based convolutional neural network ensemble classifier. In: 2024 IEEE international conference on electronics, computing and communication technologies. CONECCT, IEEE; 2024, p. 1–6.
[60] Kirubai JCD, Priscila SS. MLP-based defense mechanisms against cyber crime: Insights from dataset-driven attack prevention strategies. In: 2024 international conference on data science and network security. ICDSNS, IEEE; 2024, p. 01–6.
[61] Chan KY, Abu-Salih B, Qaddoura R, Al-Zoubi A, Palade V, Pham D-S, et al. Deep neural networks in the cloud: Review, applications, challenges and research directions. Neurocomputing 2023;545:126327.
[62] Raza MS, Arif H, Errapotu SM, Gonzalez V. Comprehensive analysis of MLP and ensemble learning approaches for intrusion detection using CICIDS2017 dataset. In: 2024 IEEE future networks world forum. FNWF, IEEE; 2024, p. 476–82.
[63] Kim J, Kim H, Kim H, Lee D, Yoon S. A comprehensive survey of deep learning for time series forecasting: Architectural diversity and open challenges. Artif Intell Rev 2025;58(7):1–95.
[64] Yavuz O, Guven E. Intention-aware intrusion detection in IoT using probabilistic models. Ad Hoc Netw 2021;122:102574.
[65] Li Y, Feng X. RNN-based cyber threat attribution for malware analysis. Comput Secur 2022;116:102651.
[66] Choudhary N, Jain A. Adversarial deep learning for host-based intrusion detection systems. Comput Secur 2022;120:102791.
[67] Ladeira WJ, Lim WM, de Oliveira Santini F, Rasul T, Frantz B, de Oliveira Rosa JC, et al. Consumer adoption of Internet of Things. J Consum Behav 2025;24(2):673–93.
[68] Ramu CK, Rao TS, Rao EUS. Attack classification in network intrusion detection system based on optimization strategy and deep learning methodology. Multimedia Tools Appl 2024;83(31):75533–55.
[69] De Neira AB, Kantarci B, Nogueira M. Distributed denial of service attack prediction: Challenges, open issues and opportunities. Comput Netw 2023;222:109553.
[70] Akinmerese O, Fasanya S, Aderotoye D, Adingupu N, Ezeoke E, Muritala R, et al. Defence against command injection attacks in a distributed network environment. Open Access Libr J 2024;11(5):1–14.
[71] Xue M, He C, Wang J, Liu W. Backdoors hidden in facial features: A novel invisible backdoor attack against face recognition systems. Peer-To-Peer Netw Appl 2021;14:1458–74.
[72] Al lelah T, Theodorakopoulos G, Reinecke P, Javed A, Anthi E. Abuse of cloud-based and public legitimate services as command-and-control (C&C) infrastructure: A systematic literature review. J Cybersecur Priv 2023;3(3):558–90.
[73] Matthijsse SR, van 't Hoff-de Goede MS, Leukfeldt ER. Your files have been encrypted: A crime script analysis of ransomware attacks. Trends Organ Crime 2023;1–27.
[74] Lim C, Jin B, Kim H. Stealth extension exfiltration (SEE) attacks: Stealing user data without permissions via browser extensions. In: Proceedings of the 40th ACM/SIGAPP symposium on applied computing. 2025, p. 1820–8.
[75] Mukhtar BI, Elsayed MS, Jurcut AD, Azer MA. IoT vulnerabilities and attacks: SILEX malware case study. Symmetry 2023;15(11):1978.
[76] Smiliotopoulos C, Barmpatsalou K, Kambourakis G. Revisiting the detection of lateral movement through sysmon. Appl Sci 2022;12(15):7746.
[77] Nobles C. The weaponization of artificial intelligence in cybersecurity: A systematic review. Procedia Comput Sci 2024;239:547–55.
[78] Bharati S, Podder P. Machine and deep learning for iot security and privacy: Applications, challenges, and future directions. Secur Commun Netw 2022;2022(1):8951961.
[79] Mallidi SKR, Ramisetty RR. Optimizing intrusion detection for IoT: A systematic review of machine learning and deep learning approaches with feature selection and data balancing. Wiley Interdiscip Rev: Data Min Knowl Discov 2025;15(2):e70008.
[80] Iranzad R, Liu X. A review of random forest-based feature selection methods for data science education and applications. Int J Data Sci Anal 2024;1–15.
[81] Zolanvari M, Teixeira MA, Gupta L, Khan KM, Jain R. WUSTL-IIOT-2021 dataset for IIoT cybersecurity research. USA: Washington University in St. Louis; 2021.
[82] Özdemir A, Polat K, Alhudhaif A. Classification of imbalanced hyperspectral images using SMOTE-based deep learning methods. Expert Syst Appl 2021;178:114986.
[83] Altalhan M, Algarni A, Alouane MT-H. Imbalanced data problem in machine learning: A review. IEEE Access 2025.
[84] Bentley JL. Multidimensional binary search trees used for associative searching. Commun ACM 1975;18(9):509–17.
[85] NVIDIA Corporation. Jetson Xavier NX product design guide. 2025, URL https://developer.nvidia.com/embedded/jetson-xavier-nx.